Design and Implementation of an IEEE 802.3 1 Gb/s Ethernet MAC/PHY Interface in Verilog-2001

Bachelor's Thesis Project (BTP-I) report submitted to

Indian Institute of Technology Kharagpur

in partial fulfilment for the award of the degree of

Bachelor of Technology

in

Electronics and Electrical Communication Engineering

by

Lingala Sai Sri Vardhan Reddy

(23EC37032)

Under the supervision of

Prof. Aneek Adhya



Department of Electronics and Electrical Communication Engineering

Indian Institute of Technology Kharagpur

Autumn Semester, 2026–27

November 2026

© 2026 Lingala Sai Sri Vardhan Reddy. All rights reserved.

DEPARTMENT OF ELECTRONICS AND ELECTRICAL COMMUNICATION ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

KHARAGPUR – 721302, INDIA



CERTIFICATE

This is to certify that the project report entitled **"Design and Implementation of an IEEE 802.3 1 Gb/s Ethernet MAC/PHY Interface in Verilog-2001"** submitted by **Lingala Sai Sri Vardhan Reddy** (Roll No. 23EC37032) to Indian Institute of Technology Kharagpur towards partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Electrical Communication Engineering is a record of bona fide work carried out by him under my supervision and guidance during Autumn Semester, 2026–27.

Prof. Aneek Adhya
Associate Professor
G.S. Sanyal School of Telecommunication
Indian Institute of Technology Kharagpur
Kharagpur – 721302, India

Date: November 2026

DECLARATION

I certify that

(a) The work contained in this report has been done by me under the guidance of my supervisor.

(b) The work has not been submitted to any other Institute for any degree or diploma.

(c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

(d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: November 2026
Place: Kharagpur

(Lingala Sai Sri Vardhan Reddy)
(23EC37032)

ABSTRACT

Name of the student	Lingala Sai Sri Vardhan Reddy
Roll No.	23EC37032
Degree for which submitted	Bachelor of Technology
Department	Department of Electronics and Electrical Communication Engineering
Report title	Design and Implementation of an IEEE 802.3 1 Gb/s Ethernet MAC/-PHY Interface in Verilog-2001
Supervisor	Prof. Aneek Adhya
Month and year of submission	November 2026

This work is part of the group's effort to build an **XGS-PON** (10 Gb/s symmetric) optical line terminal and optical network unit on a Xilinx UltraScale FPGA, towards India's first indigenous OLT/ONU prototype. Before the optical datapath can be trusted, the surrounding hardware must be tested through a processor interface built on a **MicroBlaze** soft processor and DRAM. In that bring-up the MAC reported here replaces the XGS-PON framing logic and feeds a **GTY** transceiver whose internal protocol blocks are bypassed.

This report presents the design, implementation and verification of a full-duplex **IEEE 802.3 Ethernet MAC** in vendor-neutral Verilog-2001 with GMII and MII interfaces. It replaces an encrypted commercial core whose evaluation licence had expired. The design is fourteen modules of about 2,270 lines. The transmitter is a seven-state hub controller that selects among a preamble source, a padding and payload path and a CRC tap; the receiver is a four-stage pipeline that hunts the start-of-frame delimiter, verifies the frame check sequence by residue and commits or rewinds each frame as it crosses to the host clock. Three unrelated clocks are crossed by structure: one **Gray-coded frame count** per FIFO boundary makes a frame wholly visible or wholly invisible on the far side.

Fourteen self-checking testbenches run **915 assertions with zero failures**, including a pin-to-pin loopback that returns a three-octet payload byte-identical at the padded length and shows that a single corrupted bit in flight produces no delivery. Structural assertions found three defects that value-only checking had passed. The core replaces the licensed block at zero licence cost and without any vendor primitive.

Keywords: Ethernet MAC, IEEE 802.3, CRC-32, GMII, MII, clock-domain crossing, Verilog-2001, loopback verification, XGS-PON, GTY transceiver, MicroBlaze.

ACKNOWLEDGEMENTS

I thank my supervisor, **Prof. Aneek Adhya**, for his guidance through this project, and for insisting that a design is not finished until it can be demonstrated rather than described. He is the Principal Investigator (PI) of the project *"Prototype Development for XGS-PON OLT and ONU"*, of which this work is a part, and **Dr. Amit Kumar Dutta** is its Co-PI; I thank both for the opportunity.

I am especially grateful to **Ashutosh Das**, MS (Research) scholar at IIT Kharagpur, who worked under Prof. Adhya as a Project Engineer on the same project and whom I contacted most often during this work. His explanations of what the commercial core did, and where it was used in the XGS-PON design, shaped the scope of this report.

I also thank the members of the group whose FPGA design first exposed the licensing problem, and the teaching assistants of the VLSI laboratory for their help with the simulation environment.

The open-source tools used here — the Icarus Verilog compiler and the GTKWave viewer — carried the whole verification effort, and I thank their maintainers.

Finally, I thank my family for their patience during the semester in which this work was carried out.

(Lingala Sai Sri Vardhan Reddy)

(23EC37032)

Contents

Certificate i
Declaration ii
Abstract iii
Acknowledgements iv
Contents v
List of Figures vii
List of Tables viii
List of Abbreviations ix
1 Introduction 1
1.1 Project Context . . . 1
1.2 Motivation . . . 2
1.3 Related Work . . . 2
1.4 Research Gap and Objectives . . . 2
1.5 Contributions . . . 2
1.6 Scope and Organisation . . . 2
2 The IEEE 802.3 Standard and Its Arithmetic 4
2.1 The MAC in the Protocol Stack . . . 4
2.2 Frame Format . . . 4
2.3 Preamble, Delimiter and Padding . . . 6
2.4 The Frame Check Sequence . . . 6
2.5 Inter-Frame Gap . . . 8
2.6 GMII and MII Interfaces . . . 8
3 System Architecture 9
3.1 The Top Level . . . 9
3.2 The Transmit Datapath . . . 9
3.3 The Transmit Controller . . . 12
3.4 The Receive Datapath . . . 12
3.5 Clock Domains and Reset . . . 13
4 Module Design and Waveforms 14
4.1 The Module Map . . . 14
4.2 The Padder . . . 14
4.3 The CRC Serialiser . . . 14
4.4 The Pin Stage . . . 14
4.5 The Hold-Back Octet . . . 15
4.6 Measured Waveforms . . . 15
5 Verification and Results 18
5.1 Method . . . 18
5.2 The Regression . . . 18
5.3 The Loopback Bench . . . 19
5.4 Defects Found . . . 21
5.5 Discussion . . . 21
6 Conclusion and Future Work 22
6.1 Summary . . . 22
6.2 What the Design Settles . . . 22
6.3 Limitations . . . 23
6.4 Future Work . . . 23
7 Handover and Continuation 24
7.1 What Is Delivered . . . 24
7.2 Rebuilding the Evidence . . . 24
7.3 The Interface in One Page . . . 25
7.4 Rules That Carry the Design . . . 26
7.5 What Is Proven and What Is Not . . . 27
7.6 Six Things Not To Do . . . 27
7.7 Continuing to Hardware . . . 28
8 Bibliography 29

A Delivery Reference **30**
A.1 File Inventory and Build Order 30
A.2 Parameter Reference 31
A.3 Status and Configuration Signals 31
A.4 Clause Map 32

List of Figures

1.1 Role of the MAC in the bring-up configuration . . . 1

2.1 Position of the work in the IEEE 802.3 layering . . . 4
2.2 Ethernet MAC system architecture . . . 5
2.3 Ethernet frame format . . . 5
2.4 The shared CRC-32 engine and its two uses . . . 7
2.5 Block diagram of the CRC-32 generator . . . 7
2.6 The inter-frame gap in clock cycles . . . 8
2.7 MII nibble serialisation . . . 8

3.1 Top level of the MAC . . . 9
3.2 The loopback bench as a virtual top level . . . 10
3.3 Transmit top-level architecture . . . 10
3.4 Internal structure of the transmit controller . . . 11
3.5 The seven states of the transmit controller . . . 12
3.6 The receive pipeline . . . 12
3.7 Receive controller top level and pipeline . . . 13
3.8 Clock domains and the two crossings . . . 13

4.1 The zero-slack FCS handoff . . . 15
4.2 The hold-back register in the receiver . . . 15
4.3 Transmit waveform, idle to idle . . . 16
4.4 Transmit waveform, many frames . . . 16
4.5 Transmit waveform, first frame enlarged . . . 16
4.6 Receive waveform from the testbench dump . . . 17
4.7 Receive waveform, many frames . . . 17
4.8 Receive waveform, one frame enlarged . . . 17

5.1 The loopback bench . . . 19
5.2 Loopback waveform for the payload abc . . . 19
5.3 The loopback bench with one corrupted octet . . . 20
5.4 Loopback waveform with one corrupted octet . . . 20

List of Tables

1.1 People and roles 1

2.1 The frame on the wire 4
2.2 The pin interface 8

3.1 The seven transmit states 12

4.1 The fourteen synthesised modules 14

5.1 The fourteen testbenches and their results 18
5.2 Defects found by structural assertions 21

6.1 Summary of results 22

7.1 The interface contract 25
7.2 State of the work 27
7.3 Milestones for the hardware continuation 28

A.1 Parameters 31
A.2 Status and configuration signals 31
A.3 Clause map 32

List of Abbreviations

Abbreviation	Meaning
AXI4-Stream	Advanced eXtensible Interface 4, streaming protocol
CDC	Clock-domain crossing
CRC	Cyclic redundancy check
DA	Destination address
DRAM	Dynamic random-access memory
FCS	Frame check sequence
FIFO	First-in, first-out buffer
FPGA	Field-programmable gate array
FSM	Finite-state machine
GMII	Gigabit media-independent interface
GTY	Xilinx UltraScale high-speed serial transceiver
IEEE	Institute of Electrical and Electronics Engineers
IFG	Inter-frame gap
LSB	Least-significant bit
MAC	Medium access control
MDIO	Management data input/output
MII	Media-independent interface
OLT	Optical line terminal
ONU	Optical network unit
PCS	Physical coding sublayer
PHY	Physical layer device
PMA	Physical medium attachment
PMD	Physical medium dependent
PON	Passive optical network
RTL	Register-transfer level
RX	Receive
SA	Source address
SFD	Start-of-frame delimiter
TX	Transmit
VLAN	Virtual local area network
XGS-PON	10-gigabit-capable symmetric passive optical network (ITU-T G.9807.1)

Chapter 1. Introduction

1.1 Project Context

Our research group is developing an optical access system based on the XGS-PON architecture — a passive optical network carrying 10 Gb/s symmetric traffic over fibre — on a Xilinx UltraScale FPGA. The system is the group's route to **India's first indigenous OLT** (optical line terminal) and ONU (optical network unit) prototype, so that the line-rate logic, the transceiver configuration and the board around them are designed in-house rather than bought as black boxes.

A line-rate design of this kind cannot be debugged all at once. Before the XGS-PON datapath is trusted, the rest of the hardware architecture must be shown to work: reference clocks, the DRAM, the transceiver power and clocking, and the optical front end. Testing that hardware requires a working *processor interface*: a **MicroBlaze soft processor** with DRAM, which configures the design, generates traffic and reads results back.

The work belongs to the project *"Prototype Development for XGS-PON OLT and ONU"*, whose Principal Investigator is **Prof. Aneek Adhya** and Co-PI **Dr. Amit Kumar Dutta**; day-to-day guidance came from **Ashutosh Das**, MS (Research) scholar and Project Engineer on the project. The task of this work is to supply the link layer for that interface. **An Ethernet MAC** is written from the public standard, and for bring-up it is placed where the XGS-PON MAC would sit (Fig. 1.1). Its output drives the **GTY transmitter** with the transceiver's internal protocol logic bypassed, so that the GTY is used purely as a serialiser. Known frames sent from the MicroBlaze through the MAC and the GTY then show whether the surrounding hardware — clocking, DRAM path, board and optics — behaves correctly, without the XGS-PON logic itself being a variable in the test. The people and roles are listed in Table 1.1.

Table 1.1: People and roles in the project this work belongs to.

Role	Name	Position
Principal Investigator (PI) and supervisor	**Prof. Aneek Adhya**	Associate Professor, G.S. Sanyal School of Telecommunication, IIT Kharagpur
Co-Principal Investigator (Co-PI)	**Dr. Amit Kumar Dutta**	Co-PI of "Prototype Development for XGS-PON OLT and ONU"
Mentor	**Ashutosh Das**	MS (Research) scholar, IIT Kharagpur; Project Engineer on the project
Student	Lingala Sai Sri Vardhan Reddy (23EC37032)	Author of this work; B.Tech, Electronics and Electrical Communication Engineering

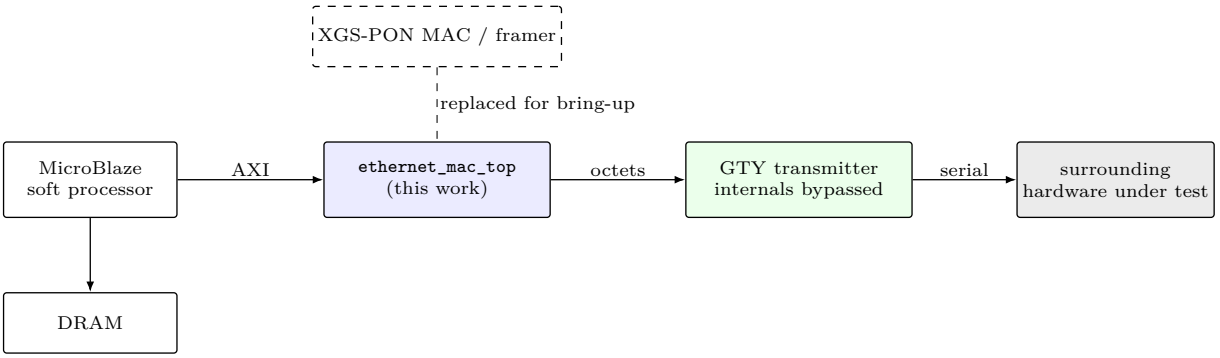


Fig. 1.1: Role of the MAC in the bring-up configuration (schematic). The MicroBlaze and its DRAM form the processor interface; the MAC stands in for the XGS-PON framing logic and feeds the GTY transmitter with its internal protocol blocks bypassed, so that the clocks, DRAM path, board and optics around it can be tested with known frames.

1.2 Motivation

Ethernet is the default interconnect of almost every digital system, and the medium access control (MAC) sublayer that makes it work is normally bought rather than built: commercial cores are encrypted, bound to vendor primitives and licensed per instance. In this group's FPGA design — the XGS-PON platform of Section 1.1 — such a **licence expired**, and the quote to replace one instance was seven to eight lakh rupees. **The premise of this work** is that a block fully specified in a public standard should not cost that much to own, so the report specifies, implements and verifies an IEEE 802.3 full-duplex MAC in vendor-neutral Verilog-2001, with no vendor primitive anywhere in the source.

1.3 Related Work

1.3.1 Commercial MAC IP

Vendor cores such as the Tri-Mode Ethernet MAC (Xilinx Inc., 2017) and the Triple-Speed Ethernet IP (Intel Corporation, 2021) cover 10, 100 and 1000 Mb/s with AXI4-Stream or Avalon host ports and full statistics. They are mature, but delivered as encrypted netlists licensed per instance, and closed to inspection — for a design whose purpose is to understand the protocol, that opacity is the problem.

1.3.2 Open-Source Cores

The OpenCores MAC (OpenCores, 2012) is genuinely open, but built around one bus and one PHY width, with no gigabit path, and verified by directed stimulus files rather than a self-checking regression.

1.3.3 Clock Crossing and CRC

Asynchronous FIFOs whose only crossing signal is a **Gray-coded pointer**, and the rule that every crossing be Gray-coded or two-flop synchronised, are due to Cummings (Cummings, 2002, 2008); Section 3.5 applies that discipline. The **frame check sequence** is a systematic cyclic code whose remainder is linear in the message, which lets one engine serve both directions (Koopman and Chakravarty, 2004).

1.4 Research Gap and Objectives

Published designs either hide their internals behind a licence, address a single bus and PHY width, or report behaviour without arguing why their clock crossings are safe. The objectives are therefore narrow:

- one full-duplex MAC serving GMII and MII from a single datapath;
- clock crossing by construction, not by timing assumption;
- correctness shown by assertions that fail when the design is wrong, including a loopback proving that a corrupted frame is never delivered.

1.5 Contributions

- **Fourteen RTL modules**, about 2,270 lines: a transmit chain, a receive chain and one wrapper, with no vendor primitive.
- **One Gray-coded frame count per FIFO boundary**, so a frame is wholly visible or wholly invisible across a clock crossing.
- **One datapath for GMII and MII**, with CRC, padder and gap timer scaled by a width strap rather than duplicated.
- **915 self-checking assertions in fourteen testbenches**, zero failures, and three real defects that value-only checking had passed.

1.6 Scope and Organisation

The scope stops at the MAC and its pin interface: no collision detection or backoff, no PAUSE, no MDIO, no address filtering, no VLAN, no board bring-up; Chapter 6 names each gap, including the MicroBlaze and GTY integration of Section 1.1, which this report prepares but does not perform. The chapters follow the work — standard and arithmetic (Chapter 2), architecture (Chapter 3), modules

and waveforms (Chapter 4), verification (Chapter 5), conclusion (Chapter 6), and finally the engineering handover (Chapter 7), which records what was delivered, how the evidence is rebuilt and where the work continues. A reference appendix then collects the file list, the parameters and the signals.

Chapter 2. The IEEE 802.3 Standard and Its Arithmetic

This chapter states the parts of IEEE 802.3 (IEEE Computer Society, 2018) that the RTL implements, and the arithmetic behind each, so that every diagram that follows reads as a consequence of a rule.

2.1 The MAC in the Protocol Stack

The MAC occupies the lower half of layer 2 and terminates at the GMII (Fig. 2.1). Above it is software; below it belong line coding, serialisation, clock recovery and the analogue front end. The block owns framing, addressing, the **frame check sequence**, padding and the inter-frame gap, and nothing else.

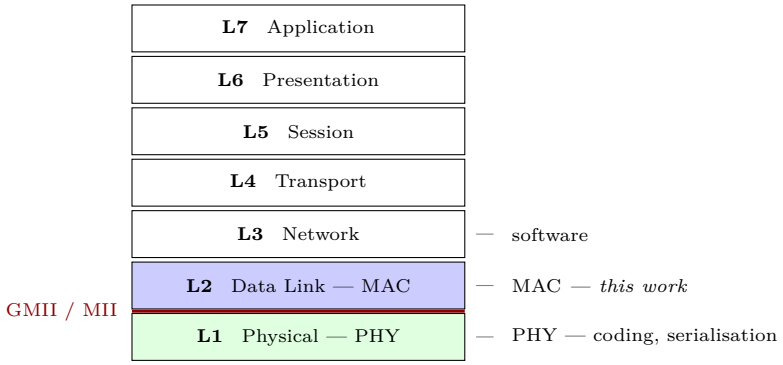


Fig. 2.1: Position of the work in the IEEE 802.3 layering. The block owns framing, addressing, the frame check sequence, padding and the inter-frame gap; coding, serialisation and clock recovery belong to the PHY, and the GMII or MII pins are the exact boundary at which the block stops.

Because that boundary is a pin interface, the block knows nothing of the medium — it drives octets and a carrier flag. That is what lets one datapath serve 10, 100 and 1000 Mb/s, and what makes the internal loopback of Section 5.3 a wiring choice rather than a mode.

The same layering, with the signals that cross each boundary, is shown in Fig. 2.2. The host reaches the MAC over AXI4-Stream, the MAC reaches the PHY over GMII or MII, and everything below that — the PCS, PMA and PMD — is the PHY's business. The figure labels the GMII/MII interface with its 1.0 Gb/s data rate; the interface clock itself is 125 MHz, one octet per cycle — $125\,\text{MHz} \times 8\ \text{bits} = 1\,\text{Gb/s}$.

2.2 Frame Format

The body carries destination and source addresses, a length or type field and the payload; the FCS covers all of it and nothing else (Fig. 2.3, Table 2.1).

Table 2.1: The frame on the wire, with the rule the RTL enforces for each field.

Field	Length	Rule implemented (clause)
Preamble	7 octets	seven `0x55` octets, bit-serial `1010…` (3.2.2)
SFD	1 octet	`0xD5`; the final two one-bits end the preamble (3.2.2)
Body	46–1500	destination, source, length/type, payload (3.2.3–3.2.6)
Pad	0–46	zeros appended so the body reaches 60 octets (3.2.7)
FCS	4 octets	CRC-32 of body and pad, complemented, low octet first (3.2.9)
IFG	96 bt	idle on the wire between frames (4.4.2)

Both limits are enforced on the padded body, because that is what the CRC covers: a body below 46 octets is a runt and is padded, one above 1500 is oversized and is dropped on receive. Addresses and the

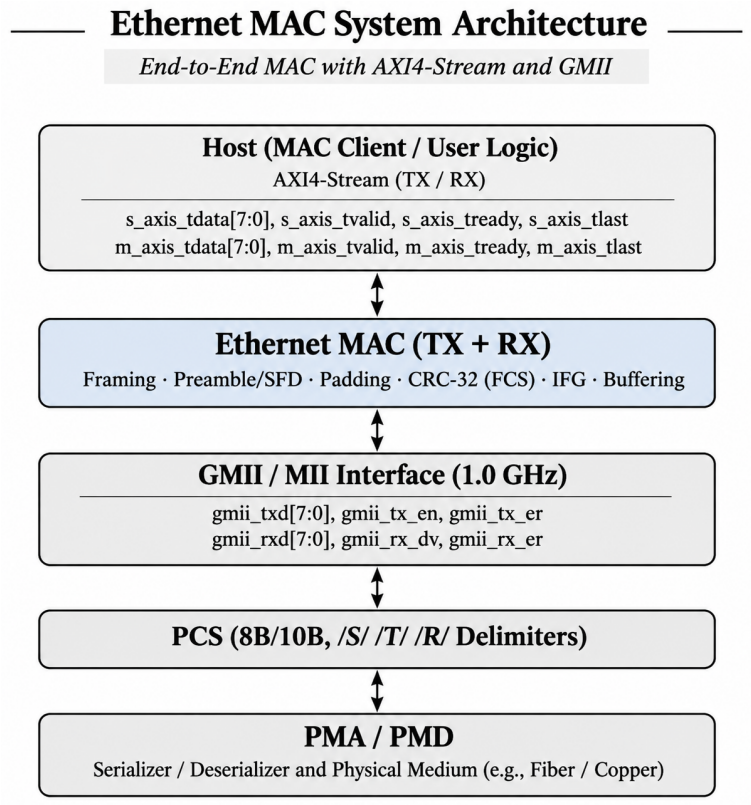


Fig. 2.2: Ethernet MAC system architecture, end to end. The host (MAC client) drives and receives AXI4-Stream (`s_axis_*` and `m_axis_*`); the MAC performs framing, preamble and SFD handling, padding, CRC-32 (FCS), inter-frame gap and buffering; the GMII/MII pins (`gmii_txd`, `gmii_tx_en`, `gmii_tx_er`, `gmii_rxd`, `gmii_rx_dv`, `gmii_rx_er`) lead to the PCS (8B/10B, start, terminate and carrier-extend delimiters), then to the PMA/PMD serialiser and the physical medium, such as fibre or copper.

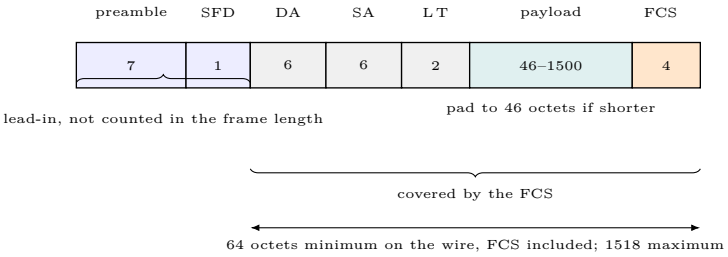


Fig. 2.3: Ethernet frame format (Clause 3.1.1). Field widths are schematic and not to scale; octet counts are inside the bar. The preamble and the SFD are excluded from the frame length and from the frame check sequence, while the pad is included in both — the asymmetry that decides the order of operations in the transmitter.

length/type field pass through uninterpreted; filtering would need a table and a management interface, both out of scope.

2.3 Preamble, Delimiter and Padding

Seven `0x55` octets give clock recovery a transition on every bit time; the delimiter is `0xD5`, the same pattern with its last two bits forced high, so a `0x55` stream can never contain that pair.

Two details in this part of the standard cause most bring-up failures:

- **The `0xAB` trap.** Octets go out **least-significant bit first**, so the bit-serial pattern `10101011` is the octet `1101_0101`: `0xD5`, not `0xAB`. A transmitter that emits `0xAB` still passes every simulation of its own output, and the link never comes up.
- **Padding order.** The 64-octet minimum keeps a station transmitting until a collision signal can return. The pad is appended *before* the CRC, so it lies inside the protected region; the receiver cannot distinguish pad from payload, and need not, because the length field delimits the payload. Taking the FCS first produces a trailer no receiver can verify.

2.4 The Frame Check Sequence

With the message as a polynomial $M(x)$ over GF(2) and the generator

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1, \tag{2.1}$$

whose coefficients are `0x04C11DB7`, the transmitted word is the complement of the remainder of $M(x)\,x^{32}$:

$$M(x)\,x^{32} = Q(x)\,G(x) + R(x), \qquad \deg R < 32, \qquad \mathrm{FCS} = \overline{R(x)}. \tag{2.2}$$

Bit-serially, with register $r_{31} \ldots r_0$ and incoming bit m,

$$f = r_{31} \oplus m, \qquad r_j \leftarrow r_{j-1} \oplus f\,g_j, \qquad r_0 \leftarrow f. \tag{2.3}$$

Reflecting the polynomial, as the standard's bit order requires, gives the byte-wise form with `0xEDB88320`:

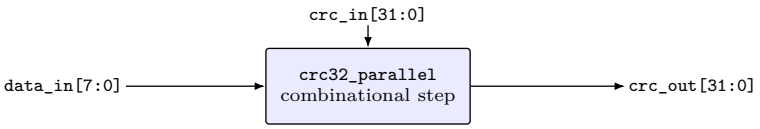
$$R_{k+1} = (R_k \gg 8) \oplus T\big[(R_k \oplus B_k) \bmod 2^8\big], \tag{2.4}$$

where T is the 256-entry table generated from the reflected polynomial. The entry $T[i]$ is exactly what eight bit-serial steps produce from a register preloaded with i, so unrolling the recurrence eight times — no memory, no clock — is the same circuit. `crc32_parallel` is that unrolled form, which is why it can be instantiated twice without any question of register ownership.

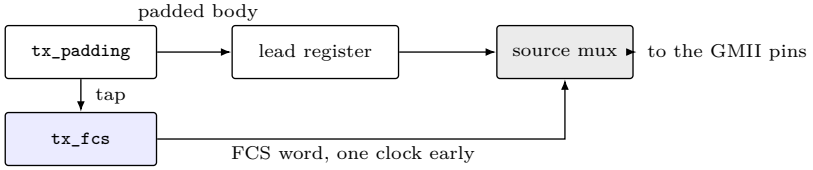
Fig. 2.4 shows the engine and its two uses. In the transmitter it is a *tap* on the padder's output, so the CRC is complete one cycle before the last body octet reaches the pins and the FCS is ready with **zero slack**. In the receiver it runs over the arriving frame while a four-deep delay line holds every octet back, so the four octets still in the line when the last FCS octet arrives are the FCS itself.

The remainder is *linear*, so one combinational step per octet suffices and nothing needs buffering; appending the transmitted FCS to the received frame leaves a **constant residue**, `0xDEBB20E3`, independent of the frame, so the receiver tests one constant instead of recomputing. The **known-answer value** `0xCBF43926` — the CRC-32 of `123456789` — catches an error in the polynomial, the reflection, the initial value or the inversion.

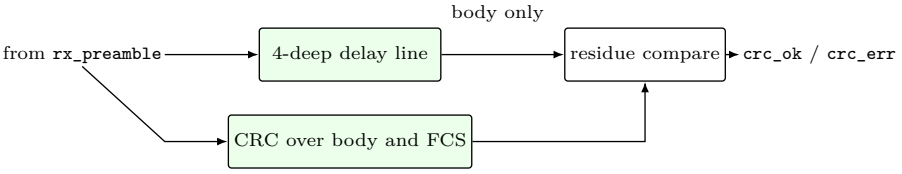
The same arithmetic, packaged as the transmit-side generator, is shown in Fig. 2.5. A small control FSM decides what the CRC register does on each clock: on `init` it is loaded with `0xFFFFFFFF`, and on `update` it takes the value of `crc32_parallel`, which folds one octet of `data_in[7:0]` into the current



reflected polynomial `0xEDB88320`: one octet in, 32 bits out



why a tap: the CRC must be finished before the last payload octet leaves



why a residue: the check is one comparison against `0xDEBB20E3`

Fig. 2.4: The shared CRC-32 engine and its two uses. (a) one combinational step: an octet and a 32-bit state in, the next state out. (b) in the transmitter the engine is a tap on the padder's output, so the register sees every octet one clock before the wire does and the inverted word is ready exactly when the multiplexer needs it. (c) in the receiver the engine runs over body and FCS while a four-deep delay line holds the FCS octets back, so the verdict is a single comparison against the residue.

32-bit state. When `frame_last` arrives the control logic raises `capture_fcs`, and the finished remainder passes through a bitwise complement to become `fcs_out[31:0]`, flagged by `fcs_valid`. The figure also records the two equivalent writings of the generator, `0x04C11DB7` for the most-significant-bit-first form and `0xEDB88320` for the reflected form that the byte-wise RTL uses.

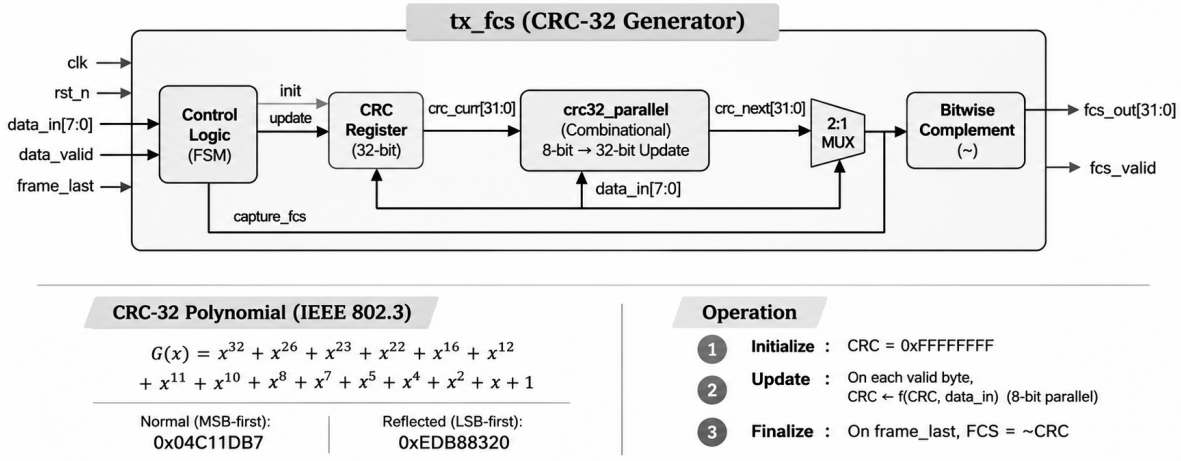


Fig. 2.5: Block diagram of the CRC-32 generator `tx_fcs`. The control logic (an FSM driven by `data_valid` and `frame_last`) issues `init` and `update` to the 32-bit CRC register, whose output `crc_curr` feeds the combinational `crc32_parallel` stage that produces `crc_next` from one octet; a 2:1 multiplexer and, on `capture_fcs`, a bitwise complement deliver `fcs_out[31:0]` and `fcs_valid`. The panel below the diagram lists the IEEE 802.3 polynomial in normal and reflected form and the three-step operation: initialise to `0xFFFFFFFF`, update on each valid octet, and complement on `frame_last`.

2.5 Inter-Frame Gap

Frames are separated by at least **96 bit times** of idle, from the last FCS bit to the next preamble; minimum frames therefore repeat every 672 bit times — 67.2 µs at 10 Mb/s, 6.72 µs at 100 Mb/s, 672 ns at 1 Gb/s (Fig. 2.6).

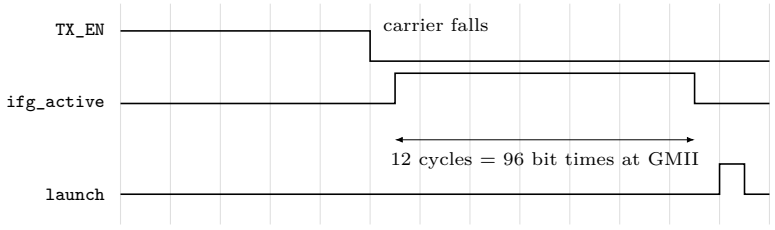


Fig. 2.6: The gap measured in clock cycles rather than bit times. `ifg_active` is high for exactly 96 bit times — twelve cycles of an 8-bit GMII datapath, twenty-four of a 4-bit MII one — and a launch is permitted in the cycle it falls. The integrated transmitter actually idles for fifteen cycles because the PHY retires its last octet one cycle late, the launch decision costs one cycle and the preamble output is registered; the minimum is a floor, not a target.

2.6 GMII and MII Interfaces

The two interfaces differ only in width — eight data bits per clock against four — and share a control encoding, so a nibble-wide port needs a serialiser but not a second datapath (Fig. 2.7, Table 2.2).

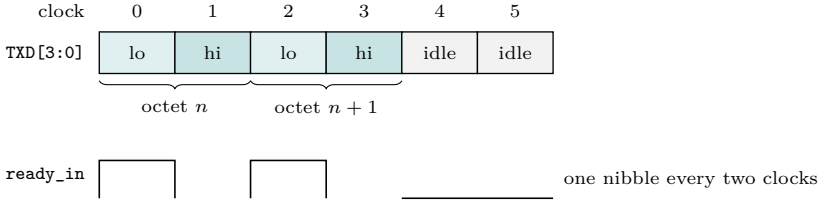


Fig. 2.7: MII nibble serialisation. The low nibble is driven first and the high nibble is owed on the next clock unconditionally; `ready_in` alternating high, low, high, low is the byte-to-nibble rate conversion seen from the controller, which is why no second datapath is needed for the narrow interface.

Table 2.2: The pin interface. The right-hand column gives the port name in the top-level module.

Signal	Direction	Port	Meaning
`GTX_CLK`	in	`gtx_clk`	125 MHz transmit clock, gigabit only
`TX_CLK`	in	`tx_clk`	25 MHz (MII) or 125 MHz (GMII) pin clock
`TXD[7:0]`	out	`TXD`	transmit data, eight bits on GMII
`TXD[3:0]`	out	`TXD`	transmit data, four bits on MII
`TX_EN`	out	`TX_EN`	data valid on the pins
`TX_ER`	out	`TX_ER`	transmit coding error, asserted on abort
`RX_CLK`	in	`rx_clk`	recovered receive clock
`RXD`	in	`RXD`	receive data, same widths
`RX_DV`	in	`RX_DV`	receive data valid; its fall ends a frame
`RX_ER`	in	`RX_ER`	receive coding error from the PHY

On MII the low nibble goes first and the high nibble is owed on the next clock unconditionally; `ready_in` alternating high, low *is* the byte-to-nibble conversion.

The clocking is asymmetric. In GMII the MAC sources `GTX_CLK` and the PHY returns `TX_CLK`; in MII the PHY sources it.

Chapter 3. System Architecture

The partition follows one rule: a module owns exactly one rule of the standard — fourteen modules in three groups, with no module reaching across another's responsibility.

3.1 The Top Level

The top level (Fig. 3.1) exposes three clocks and their resets, an AXI4-Stream slave port for frames sent, a master port for frames received, the GMII/MII pins, and a status bus of counters, error flags and link configuration.

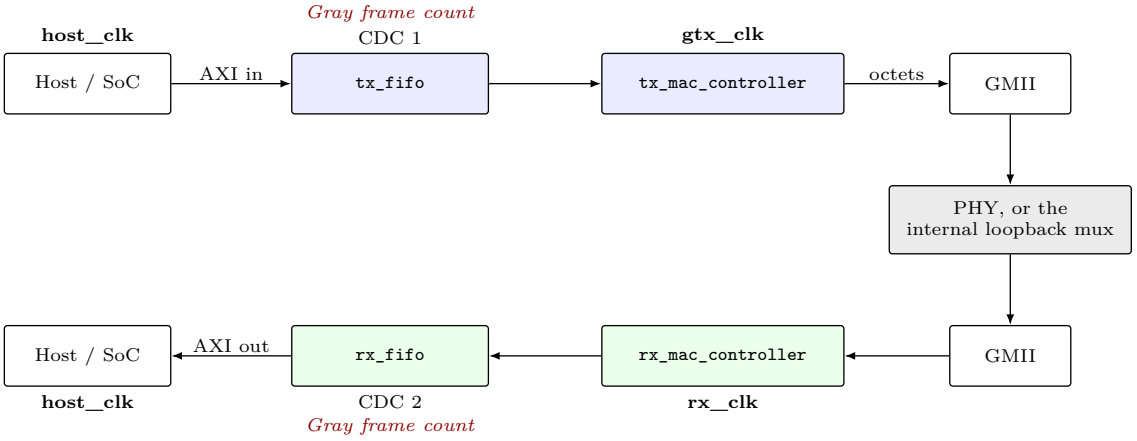


Fig. 3.1: Top level of `ethernet_mac_top`. The two FIFOs are the only places where a frame crosses a clock boundary, and each crossing carries one Gray-coded frame count and nothing else. No signal runs from the transmit row to the receive row: the halves meet only at the pins, which is what "full duplex" means in hardware.

Fig. 3.2 places the same two halves in the environment in which every waveform of Chapters 4 and 5 was recorded. The transmitter and receiver are joined at the pins through a corruptor with no state of its own: when the current octet is the one selected by `chan_flip_idx`, counted from the rise of carrier (`carrier_cyc`), the channel flips the chosen bits or forces `rx_er`; otherwise it passes the octet unchanged. With the corruptor idle the bench is a plain loopback; with it enabled the same bench becomes the corruption sweep of Section 5.3.

The halves meet only at the pins, which lets each be verified against a model of the other and makes the loopback a wiring choice rather than a mode; every readable counter is therefore exported explicitly.

3.2 The Transmit Datapath

The transmitter is a *star*, not a pipeline: a **seven-state controller** sits at the centre and every other block is a source it selects from (Fig. 3.3, Fig. 3.4). Payload crosses into the transmit clock in a **store-and-forward FIFO**, is padded and forwarded octet by octet; the CRC is a tap on the padder's output, the preamble is an alternative source, and the gap timer is a side timer with no data ports.

Fig. 3.3 gives the transmit half at pin level. Everything left of the FIFO runs on the host clock and everything right of it on the wire clock, so the figure is also a map of where each port lives. The frame formatter groups the four blocks that build a frame — preamble (eight octets), padding, FCS and the twelve-octet gap — and the controller above them reports `frame_sent_out`, `underrun_out` and `tx_busy_out`. Two readings need care. First, the figure's "≥46 B" for the padder is the minimum of the *payload* field; the RTL pads the whole body, header included, to 60 octets, which is the same rule counted

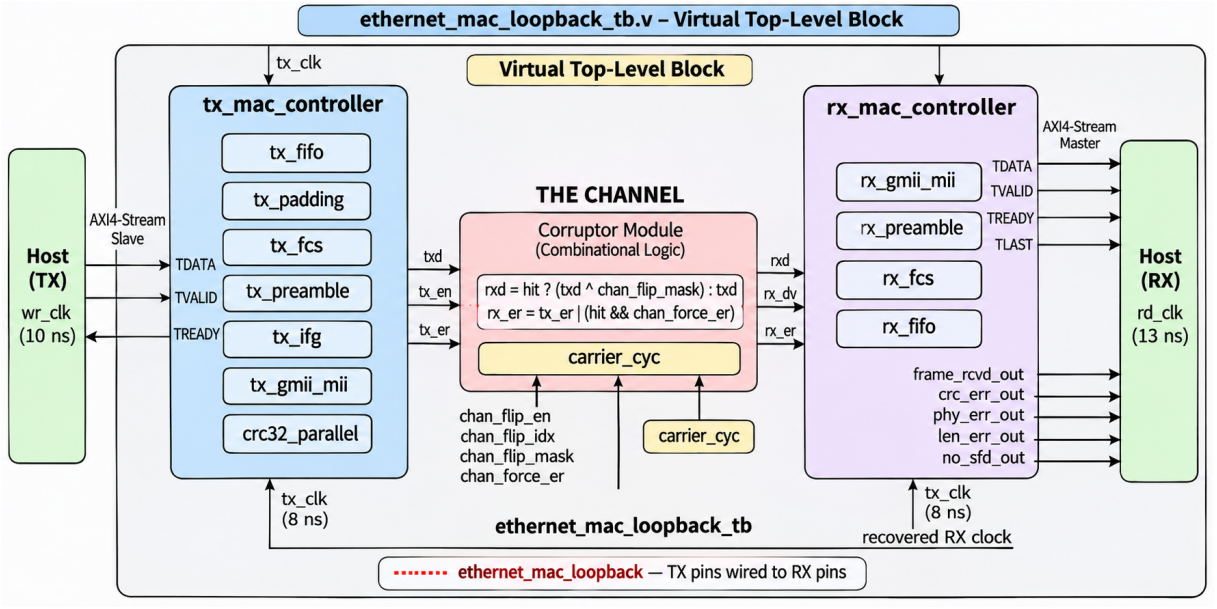


Fig. 3.2: The two halves inside the loopback bench `ethernet_mac_loopback_tb.v`, drawn as a virtual top level. The host (TX) writes frames over AXI4-Stream on `wr_clk` (10 ns) into `tx_mac_controller`, whose six sub-blocks and `crc32_parallel` are listed inside it. Its `txd`, `tx_en` and `tx_er` cross "the channel", a purely combinational corruptor that can XOR a chosen mask into one octet (`rxd = hit ? (txd ^ chan_flip_mask) : txd`) or force `rx_er`, under the controls `chan_flip_en`, `chan_flip_idx`, `chan_flip_mask` and `chan_force_er`. `rx_mac_controller` then delivers frames over AXI4-Stream on `rd_clk` (13 ns) to the host (RX) and raises `frame_rcvd_out`, `crc_err_out`, `phy_err_out`, `len_err_out` and `no_sfd_out`. The 8 ns wire clock drives both halves.

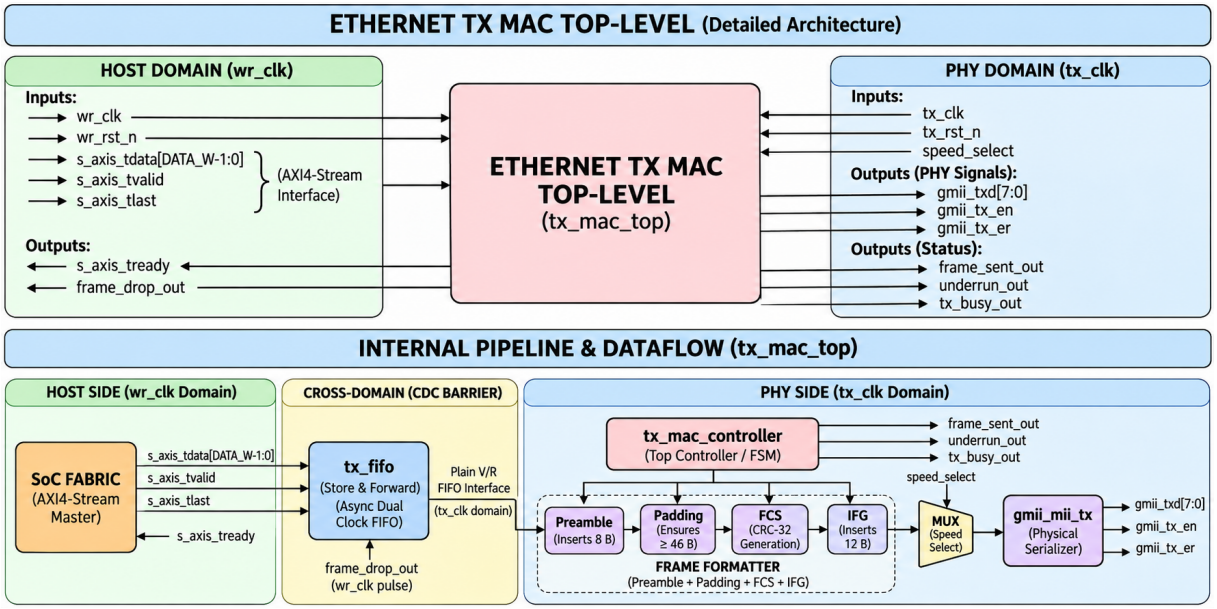


Fig. 3.3: Detailed architecture of the transmit top level `tx_mac_top`. The host domain (`wr_clk`) carries the AXI4-Stream slave port `s_axis_tdata/tvalid/tlast`, `s_axis_tready` and the `frame_drop_out` pulse; the PHY domain (`tx_clk`) carries the clock, reset, `speed_select`, the pins `gmii_txd`, `gmii_tx_en` and `gmii_tx_er`, and the status outputs `frame_sent_out`, `underrun_out` and `tx_busy_out`. Below, the SoC fabric writes into `tx_fifo`, the store-and-forward asynchronous FIFO that is the only clock-domain crossing, whose plain valid/ready output feeds `tx_mac_controller` and its frame formatter (preamble, padding, FCS and gap), then the speed-select multiplexer and `gmii_mii_tx`.

from the destination address. Second, the left-to-right order of the formatter blocks is functional, not temporal: the controller selects among them, as Fig. 3.4 shows.

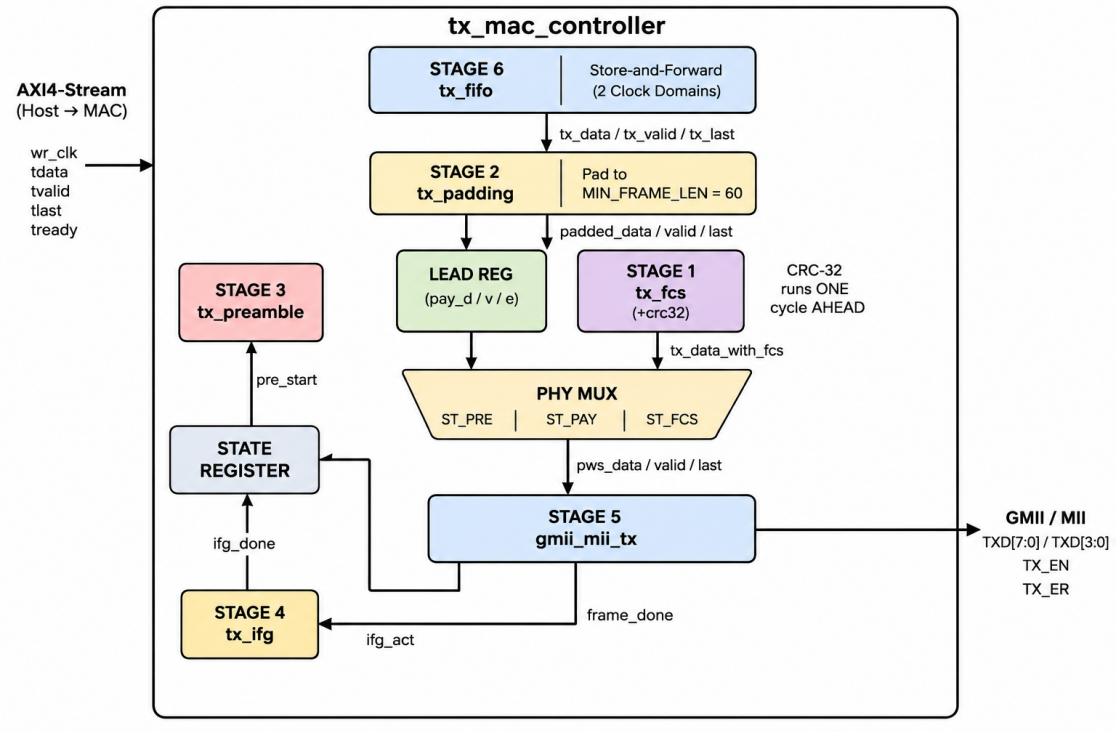


Fig. 3.4: Internal structure of `tx_mac_controller`. AXI4-Stream data enters `tx_fifo` (stage 6, store-and-forward across two clock domains) and then `tx_padding` (stage 2, padding to `MIN_FRAME_LEN = 60`). The padded stream feeds both a one-stage lead register and `tx_fcs` (stage 1, with `crc32`), which runs one cycle ahead. The PHY multiplexer selects preamble (`ST_PRE`), payload (`ST_PAY`) or FCS (`ST_FCS`) and passes `pws_data/valid/last` to `gmii_mii_tx` (stage 5), which drives `TXD`, `TX_EN` and `TX_ER`. The state register starts `tx_preamble` (stage 3) with `pre_start`, and `frame_done` from stage 5 starts `tx_ifg` (stage 4), whose `ifg_done` returns to the state register.

Fig. 3.4 opens the controller. The stage numbers are labels of the modules, not their order in the dataflow: the state register is the hub, the preamble generator and the gap timer hang off it as a source and a timer, and the datapath is padding, then the lead register and the CRC tap side by side, then the multiplexer. The CRC runs **one cycle ahead** of the lead register, so by the time the last payload octet is selected the four FCS octets are already available to the multiplexer.

Three invariants hold it together:

- **pad precedes the FCS**, because pad octets are inside the covered field;
- preamble and delimiter **bypass the padder and the CRC**, because they are outside the frame length; and
- the lead register is **one stage deep**, because that is what makes the FCS arrive on time.

3.3 The Transmit Controller

Five states carry the forward path and two exist for the PHY's sake (Fig. 3.5, Table 3.1). `ST_WAIT` covers the cycle in which the pin stage still holds the previous octet; `ST_FLSH` drains an aborted frame so its `TX_ER` cannot land on the next frame's first octet.

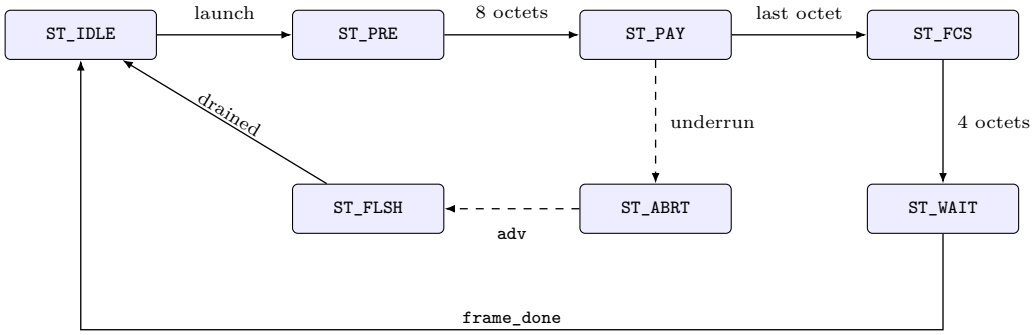


Fig. 3.5: The seven states of the transmit controller. The top row is the payload path; `ST_WAIT`, `ST_ABRT` and `ST_FLSH` exist so that the wire stays clean at the end of a frame, and the dashed path is taken only when the host stops supplying octets in the middle of a frame.

Table 3.1: The seven transmit states, what each does and the condition that ends it.

State	Function	Exit
`ST_IDLE`	wait for a committed frame and for the gap to expire	`launch`
`ST_PRE`	emit eight preamble octets, `0x55`×7 then `0xD5`	8th octet handed over
`ST_PAY`	stream body and pad, tapping the CRC engine on the way	last body octet
`ST_FCS`	emit the four latched CRC octets, low octet first	4th octet
`ST_WAIT`	one dead cycle while the PHY retires the last octet	`phy_frame_done`
`ST_ABRT`	terminate an underrun frame by asserting `TX_ER`	`adv`
`ST_FLSH`	drain the aborted frame before re-arming	`pad_eof`

3.4 The Receive Datapath

The receiver is a **four-stage pipeline**: `rx_gmii_mii` samples the pins and holds one octet back, `rx_preamble` hunts for the delimiter, `rx_fcs` checks the residue and strips four octets, and `rx_fifo` commits or rewinds whole frames across to the host clock (Fig. 3.6, Fig. 3.7).

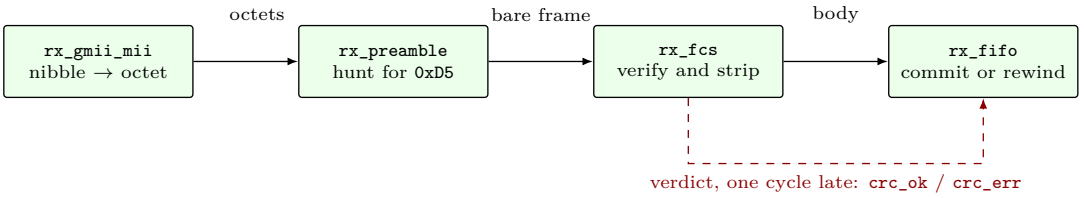


Fig. 3.6: The receive pipeline. Each stage removes exactly one thing — the nibble serialisation, the lead-in, the four FCS octets, the whole frame if it failed — and the verdict travels as a separate two-wire bus that arrives one cycle after the last body octet, which is why the FIFO must hold the frame invisible until it is known to be good.

Fig. 3.7 is the receive counterpart of Fig. 3.3: the top level on the two clock domains above and the four-stage pipeline below, with the clock crossing sitting between `rx_fcs` and the AXI4-Stream interface. The verdict leaves `rx_fcs` on a separate path to the status logic, which is the same one-cycle-late bus drawn in Fig. 3.6.

The **hold-back octet** is forced by the interface: the end of a frame is known only when `RX_DV` falls, so the first stage keeps the most recent octet and publishes it a cycle later with `last_out` high. **Commit-or-rewind** then makes a bad frame invisible: the count advances only when the residue check passes.

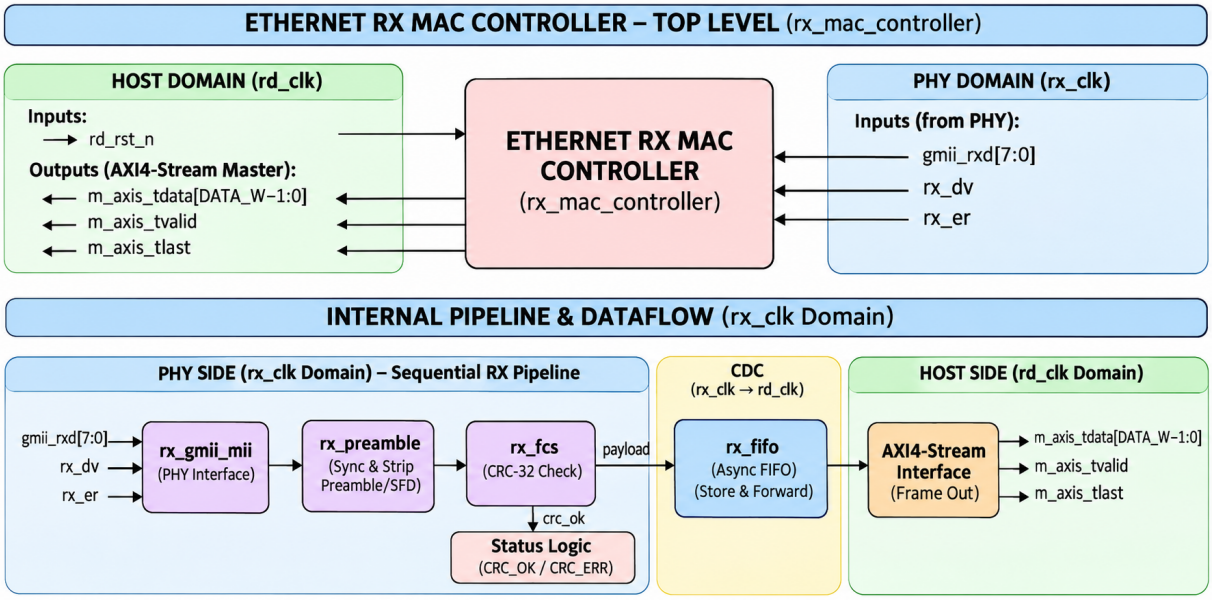


Fig. 3.7: Top level and internal pipeline of `rx_mac_controller`. In the PHY domain (`rx_clk`) the inputs `gmii_rxd[7:0]`, `rx_dv` and `rx_er` pass through `rx_gmii_mii`, `rx_preamble` (synchronise and strip preamble and SFD) and `rx_fcs` (CRC-32 check); `rx_fcs` sends the payload on and the verdict `crc_ok` to the status logic (`CRC_OK`/`CRC_ERR`). `rx_fifo`, the store-and-forward asynchronous FIFO, carries the frame from `rx_clk` to `rd_clk`, and the AXI4-Stream interface in the host domain presents it on `m_axis_tdata`, `m_axis_tvalid` and `m_axis_tlast`.

3.5 Clock Domains and Reset

Three clocks are unrelated by construction (Fig. 3.8): host (10 ns), wire transmit (8 ns) and receive (13 ns). Both FIFOs cross *one Gray-coded frame count and nothing else* (Cummings, 2002, 2008).

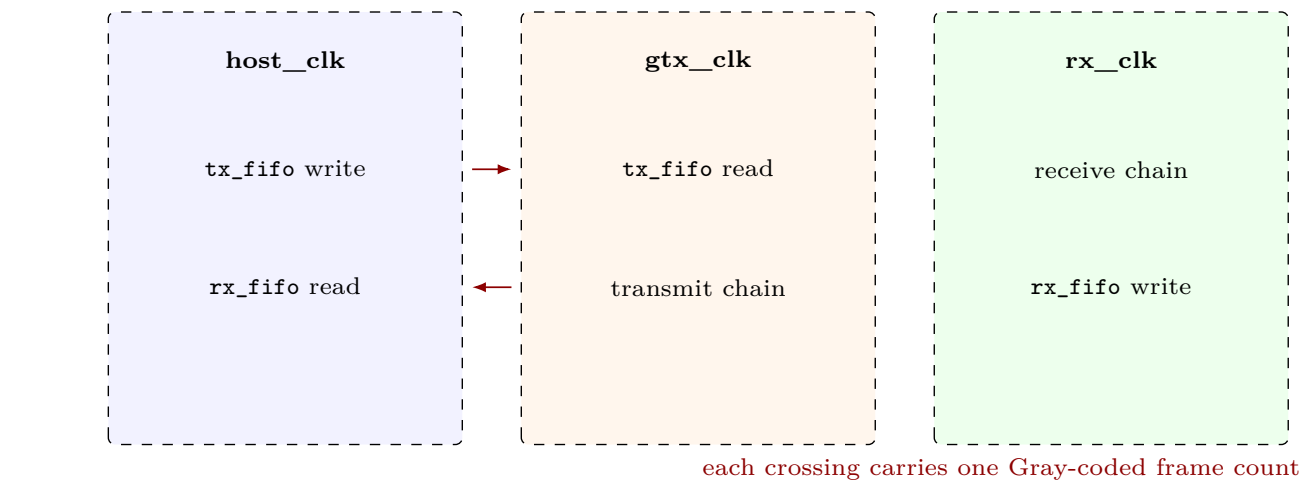


Fig. 3.8: The three clock domains and the only two crossings between them. In internal loopback `rx_clk` is driven from `gtx_clk` through a constrained clock multiplexer, because there is no PHY to source it.

Why only a count may cross. A Gray code is safe only for a quantity that changes by one and never rewinds; the committed frame count satisfies that, while byte pointers jump at a frame boundary and rewind on a discard. Hence the design's central claim: a frame is **wholly visible on the far side or wholly invisible**.

Chapter 4. Module Design and Waveforms

Nine of the fourteen modules fit in one sentence each. The five that carry the difficulty are described below, each with the diagram or timing that makes its contract visible.

4.1 The Module Map

Table 4.1 lists all fourteen modules in dataflow order, with their clock domain and contract.

Table 4.1: The fourteen synthesised modules, in dataflow order.

Module	Domain	Contract
`tx_fifo`	host/gtx	store-and-forward CDC; a frame is visible only when complete
`tx_padding`	gtx_clk	pad the body to 60 octets with zeros, before the CRC
`tx_fcs`	gtx_clk	accumulate the CRC, latch the inverted word on the falling edge
`tx_preamble`	gtx_clk	seven `0x55` then `0xD5`, as a parallel source
`tx_ifg`	gtx_clk	hold the wire idle for 96 bit times, scaled to the datapath width
`tx_gmii_mii`	tx_clk	serialise octets or nibbles onto the pins with `TX_EN`, `TX_ER`
`tx_mac_controller`	gtx_clk	the hub: 7-state FSM, source mux, launch decision
`crc32_parallel`	—	one combinational CRC-32 step, shared by both directions
`rx_gmii_mii`	rx_clk	sample the pins, derive end of frame, latch `RX_ER`
`rx_preamble`	rx_clk	hunt for `0xD5`, discard lead-in, tolerate a short preamble
`rx_fcs`	rx_clk	verify by residue, strip four octets, enforce length limits
`rx_fifo`	rx/host	commit on a good verdict, rewind on a bad one, cross to the host
`rx_mac_controller`	—	a wrapper with no state: four stages and a verdict bus
`ethernet_mac_top`	three	the full-duplex wrapper, loopback mux, status bus

4.2 The Padder

The padder passes payload through, counts octets, and when end-of-frame arrives before the body reaches 60 octets it holds `valid_out` high and emits `0x00` until the count is met — so a one-octet payload leaves it as the sixty-octet body of Fig. 2.3. **The one-octet case** asserts `sof` and `eof` on the same cycle; an earlier version tested only `sof` and hung, with no signal wrong and no value incorrect, which is why the edge-case bench detects the stall rather than a bad octet.

4.3 The CRC Serialiser

The engine is a tap, not a stage, so its result is ready one octet after the last payload octet. Latching the inverted word on the *falling* edge holds the four FCS octets stable for the whole of the next cycle, which is what lets the controller shift them out with no buffering and no stall; Fig. 4.1 traces the three cycles this depends on.

Zero slack. Every other stage could be a cycle deeper without changing anything visible; this one cannot, because the wire has no elastic buffer.

4.4 The Pin Stage

`tx_gmii_mii` holds the octet on the pins and drives `TX_EN` from a registered valid flag, so pin timing is set by one register rather than by the depth of the logic behind it; on MII that register is loaded once

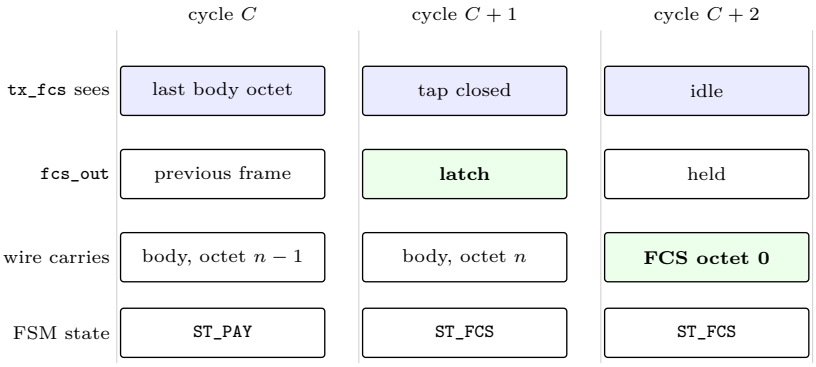


Fig. 4.1: The zero-slack FCS handoff, read left to right. The CRC register consumes the last body octet in cycle C, while the wire is still carrying the octet before it; the inverted word is latched on the falling edge in cycle $C+1$, which is the cycle in which the last body octet reaches the pins; the first FCS octet therefore follows in cycle $C+2$ with no idle cycle. Removing the lead register moves the whole chain one cycle late and ships the previous frame's trailer.

per octet and shifted out as two nibbles (Fig. 2.7). **The high nibble is unconditional**: once the low nibble is on the wire the high one is owed on the next clock, and gating it on `valid_in` emits half an octet and shifts every remaining nibble by four bits while length, carrier pulse and nibble count all still look correct. Only the far-end CRC catches it.

4.5 The Hold-Back Octet

GMII has no last-octet marker: `RX_DV` simply falls. A complete octet is therefore parked in a **hold-back register** and published with the next octet, or on the falling edge with `last_out` high (Fig. 4.2).

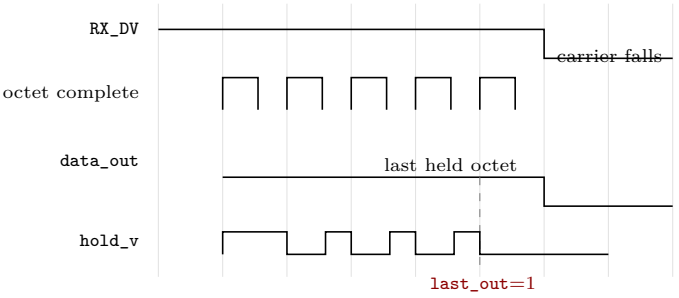


Fig. 4.2: The hold-back register in the receiver. The octet that is complete when carrier falls is published by the state that observes the fall, with `last_out` high — the only way to mark end of frame on GMII. Publishing it one cycle later, in a dedicated end state, inserts an idle cycle inside the frame and corrupts the digest.

Publishing it one cycle later inserts an idle cycle inside the frame; that version passed 41 functional checks and was still wrong, because `rx_fcs` advances the CRC on every valid cycle.

4.6 Measured Waveforms

Fig. 4.3 fixes the transmit behaviour of one frame from idle back to idle, and Fig. 4.6 the receive behaviour from carrier up to a committed frame. Both schematics use the same octet scale, so lead-in, body and pad, FCS and gap can be read against each other at opposite ends of the design. Each is followed by the recorded simulation it summarises: a long capture over many frames, and an enlarged capture of one frame in which every signal transition can be read.

The two captures give the transmit behaviour its three timing facts. The frame does not start until it is complete in the FIFO: `launch` follows `s_axis_tlast` by a few cycles, never precedes it, which is the **store-and-forward rule**. The carrier then stays high from the first preamble octet to the last FCS octet, about 72 octets at 8 ns, while the input side needed 46 octets at 10 ns, so the two sides of the FIFO plainly run on **unrelated clocks**. Finally, the FCS follows the padded body without a gap, the handoff of Fig. 4.1.

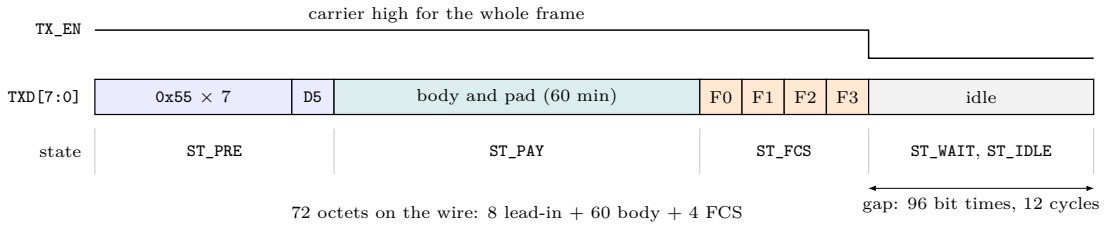


Fig. 4.3: Transmit waveform, idle to idle, from the value-change dump of `tx_mac_controller_tb` and `tx_gmii_mii_tb`. The carrier rises with the first preamble octet and falls with the last FCS octet; the four FCS octets follow the padded body without an idle cycle, which is the handoff of Fig. 4.1; the gap then runs before the controller may launch again.

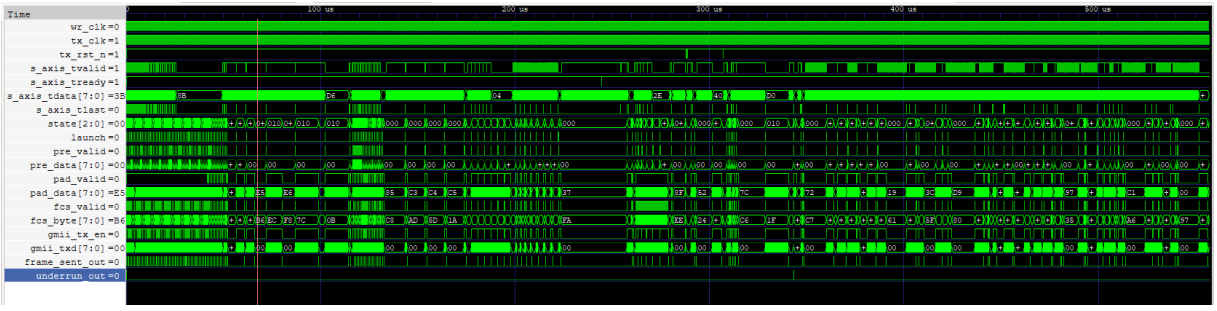


Fig. 4.4: Simulated transmit-side waveform over about 540 µs, with many frames of different lengths. Top to bottom: `wr_clk`, `tx_clk`, `tx_rst_n`, the AXI4-Stream input (`s_axis_tvalid`, `s_axis_tready`, `s_axis_tdata`, `s_axis_tlast`), the controller `state[2:0]` and `launch`, the three sources `pre_data`, `pad_data` and `fcs_byte` with their valid strobes, and the outputs `gmii_tx_en`, `gmii_txd`, `frame_sent_out` and `underrun_out`. Each burst of `s_axis_tvalid` is followed, one frame later, by a window of `gmii_tx_en`; `frame_sent_out` pulses once per frame and `underrun_out` pulses once, near 345 µs.

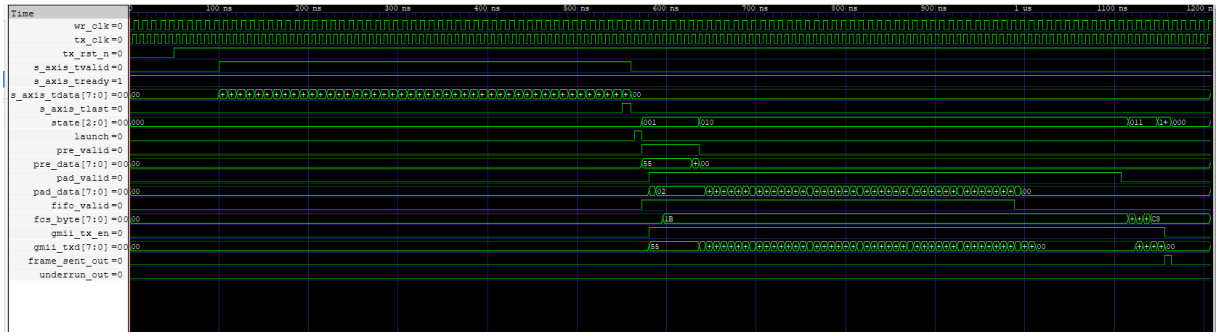


Fig. 4.5: Enlarged view of the first frame of Fig. 4.4, 0 to 1.2 µs. `tx_rst_n` is released near 50 ns; `s_axis_tvalid` rises at 100 ns and 46 octets are accepted on `wr_clk` until `s_axis_tlast`. `launch` pulses near 570 ns and `state` steps 000, 001 (ST_PRE), 010 (ST_PAY), 011 (ST_FCS) and back to 000. `pre_data` shows 0x55, `pad_data` the body followed by zero padding, `fcs_byte` the FCS octets, and `gmii_tx_en` brackets the whole frame on `gmii_txd`. `frame_sent_out` pulses at the end and `underrun_out` stays low.

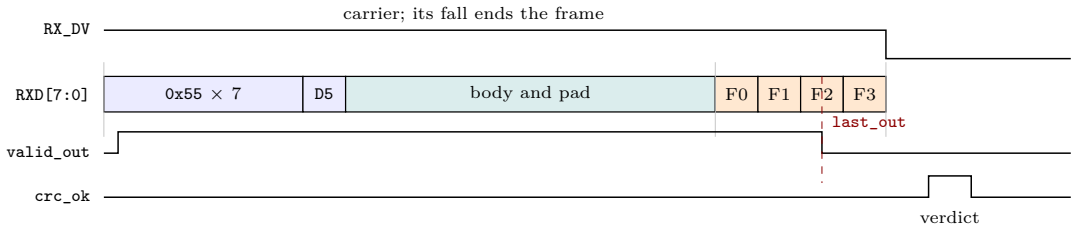


Fig. 4.6: Receive waveform from the dump of `rx_mac_controller_tb`. The lead-in is discarded, the body is published one cycle behind the pins because of the hold-back register of Fig. 4.2, and the four FCS octets are absorbed by the delay line. `crc_ok` rises one cycle after the last FCS octet; only then does the committed count move and the frame appear on `m_axis`, which is what makes a rejected frame invisible to the host.

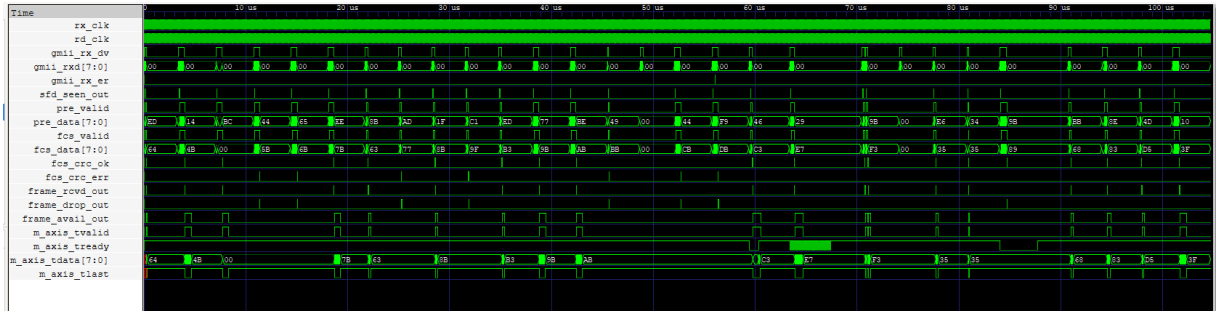


Fig. 4.7: Simulated receive-side waveform over about 100 µs, driving many frames into `rx_mac_controller`. Signals, top to bottom: `rx_clk`, `rd_clk`, `gmii_rx_dv`, `gmii_rxd`, `gmii_rx_er`, `sfd_seen_out`, the preamble-stage output (`pre_valid`, `pre_data`), the FCS-stage output (`fcs_valid`, `fcs_data`, `fcs_crc_ok`, `fcs_crc_err`), the status pulses `frame_rcvd_out`, `frame_drop_out` and `frame_avail_out`, and the host-side `m_axis` bus. Frames with a good verdict (`fcs_crc_ok`, `frame_rcvd_out`) appear on `m_axis_tdata`; frames with a bad verdict (`fcs_crc_err`, `frame_drop_out`) do not. Around 60 to 85 µs `m_axis_tready` is held low and delivery pauses, then resumes.

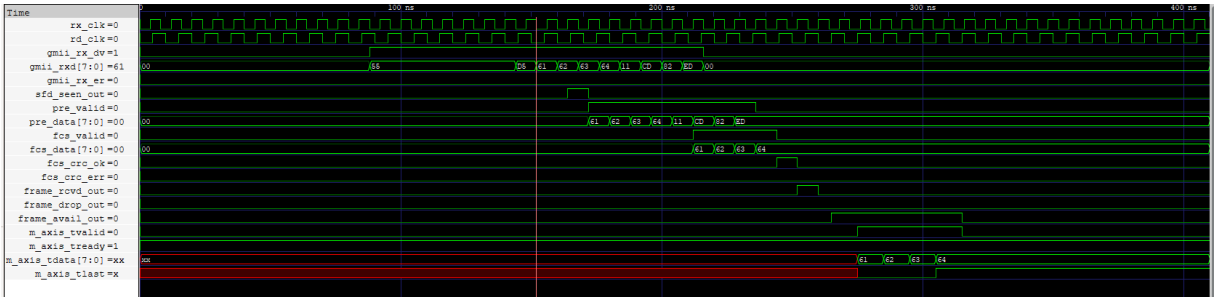


Fig. 4.8: Enlarged view of one received frame, 0 to 400 ns. `gmii_rx_dv` rises near 100 ns; `gmii_rxd` carries `0x55` lead-in, the delimiter `0xD5`, then `0x61`, `0x62`, `0x63`, `0x64` ("abcd") and the FCS octets `0x11`, `0xCD`, `0x82`, `0xED`. `sfd_seen_out` pulses at the delimiter; `pre_data` then shows all eight octets without the lead-in, and `fcs_data` only the four body octets. `fcs_crc_ok` pulses after the last FCS octet, followed by `frame_rcvd_out`, `frame_avail_out` and `m_axis_tvalid`, and `m_axis_tdata` delivers `0x61` to `0x64` with `m_axis_tlast` on the last octet.

Read together, the two captures show the receive rules in order. The delimiter is found, the lead-in is discarded, the four FCS octets are removed from the stream, and only after the verdict does the frame become visible to the host — on the slower, unrelated `rd_clk`. In the long capture the same pipeline accepts good frames and **silently discards bad ones**, with delivery paused and resumed by `m_axis_tready` without loss.

Chapter 5. Verification and Results

5.1 Method

Verification is self-checking: every bench prints `PASS` or `FAIL` per assertion plus a summary, and the regression is a shell loop over fourteen benches compiled by Icarus Verilog (Williams, 2023), with traces read in GTKWave (Bybell, 2023) only when a check fails. No bench needs a human to adjudicate a waveform, which is what makes re-running it after every change worthwhile.

Three habits did most of the work:

- **Structural assertions** sit beside every value check, because a value can be right and still arrive in the wrong cycle.
- **An independent oracle** anchors the CRC to known answers.
- **Fault injection** corrupts preamble, body and FCS octets separately, because the three fail differently: a corrupted lead-in raises `no_sfd`, a corrupted body `crc_err`.

Stimulus is driven on the falling edge and sampled by rising-edge monitors, so no check races the design, and every bench ends with a reset test.

5.2 The Regression

Table 5.1 lists the fourteen benches with their level and results, re-run on the delivered RTL.

Table 5.1: The fourteen testbenches and their results on the delivered RTL: 915 checks, zero failures.

Bench	Level	Checks	What it establishes
`tx_fcs_tb`	unit	16	known answer, random frames, backpressure, reset
`tx_fifo_tb`	unit	115	store-and-forward, CDC, overflow, pointer invariants
`tx_ifg_tb`	unit	57	gap length in GMII and MII, strap latching, 10 Mb/s
`tx_padding_tb`	unit	28	pad boundaries, a one-octet frame, freeze, reset
`tx_preamble_tb`	unit	56	burst contents, backpressure, re-launch
`tx_gmii_mii_tb`	unit	94	GMII and MII transmit, underrun abort, `TX_ER`
`tx_mac_controller_tb`	chain	110	full transmit chain against a receiver model
`tx_edge_case_tb`	chain	47	runt, size boundaries, oversize, resets, starvation
`rx_gmii_mii_tb`	unit	44	pin sampling, framing, contiguity, error latching
`rx_preamble_tb`	unit	49	short, zero and long preambles, junk lead-in
`rx_fcs_tb`	unit	42	residue, error detection, FCS strip, length limits
`rx_fifo_tb`	unit	56	commit and rewind, overflow, CDC, reset
`rx_mac_controller_tb`	chain	95	full receive chain, pins to AXI
`ethernet_mac_loopback_tb`	system	106	transmitter to receiver at the pins, corruption sweep
total		**915**	**0 failures**

Unit benches exercise one module, chain benches connect a whole direction against a model of the other end, and the system bench connects the two halves at the pins. The distribution is deliberate: the FIFO and the controllers carry the most checks, because that is where a mistake hides from a value trace, while the preamble generator carries few, because its output is a constant. The two chain benches and the system bench account for 311 checks, a third of the total, and they are the ones that catch a disagreement between two modules that are each individually correct.

5.3 The Loopback Bench

The decisive bench connects the halves at the GMII pins and drives each from an independent clock, so both crossings are exercised in opposite directions in one run (Fig. 5.1). A three-octet payload enters as `abc`, leaves as a legal sixty-octet frame — three payload, fifty-seven pad, four FCS, behind an eight-octet lead-in — and returns byte-identical.

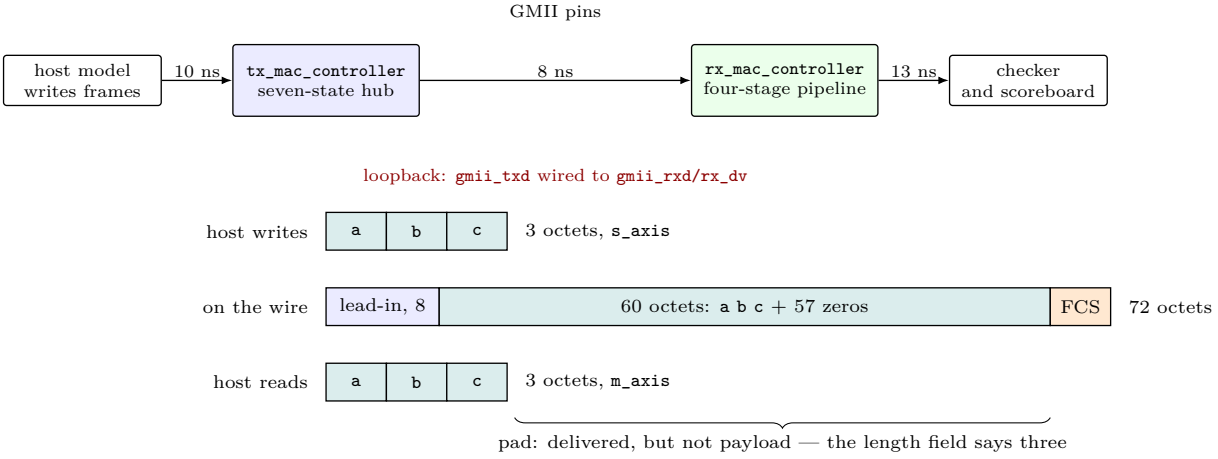


Fig. 5.1: The loopback bench. (a) the two controllers connected at the pins, with the wire clock feeding both halves as it would in a real link while the two host clocks stay unrelated to it and to each other, so both FIFO crossings are exercised in opposite directions in a single run. (b) the payload ladder: three octets in, a legal seventy-two-octet frame on the wire, the same three octets out, with the pad delivered but excluded from the payload by the length field.

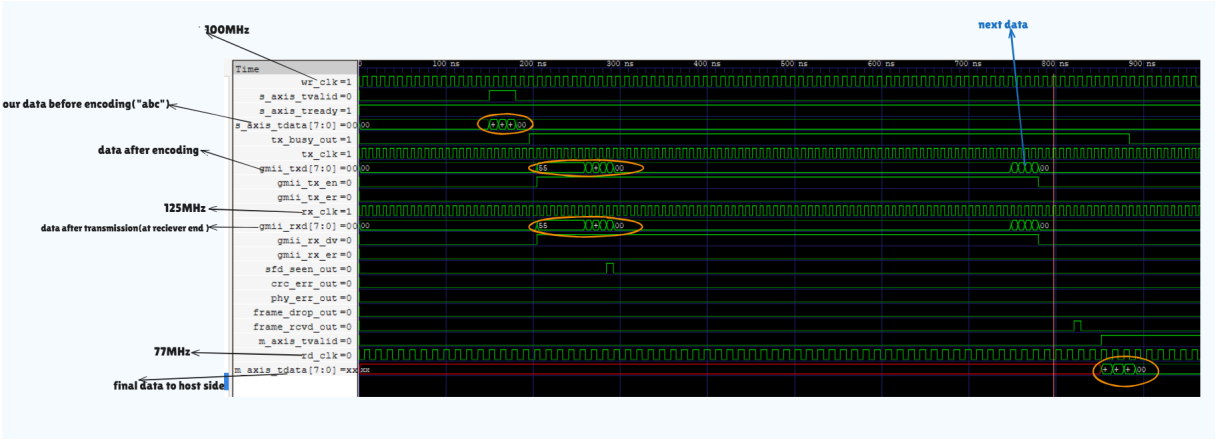


Fig. 5.2: Loopback waveform for the payload `abc`, annotated. The host writes `abc` on `s_axis_tdata` (`wr_clk`, 100 MHz) at about 150 ns; `tx_busy_out` rises and `gmii_txd` (`tx_clk`, 125 MHz) carries the lead-in `0x55`, the padded body and, just before `gmii_tx_en` falls near 780 ns, the four FCS octets. `gmii_rxd` repeats the same octets with `gmii_rx_dv`; `sfd_seen_out` pulses at the delimiter, and `crc_err_out`, `phy_err_out` and `frame_drop_out` stay low. `frame_rcvd_out` pulses near 850 ns and `m_axis_tdata` (`rd_clk`, 77 MHz) then returns the three octets `abc`.

Fig. 5.2 shows the whole path in one picture. The frame is on the pins for about 580 ns, which is 72 octets at 8 ns, and the receiver reproduces it octet for octet. The payload returns about 70 ns after the carrier falls, the time the verdict and the clock crossing take, and the interval during which the three clocks (100, 125 and 77 MHz) are all visibly independent.

Because the halves are joined where they would meet in a system, there is no model of the far end that could be wrong in the same way as the design, and no way to tune one side to the other. The clock periods are chosen so no edge lines up for long — 10 ns write, 13 ns read, 8 ns wire — so the synchronisers never settle into a repeating pattern within a frame.

A **corruption sweep** then flips a single bit at every position in flight and asserts that *nothing* is delivered: not a wrong frame, not a partial one, not one carrying a warning (Fig. 5.3). That is the assertion which

separates a design that computes a CRC from one that acts on it, and it is why the **commit-or-rewind discipline** is worth its complexity.

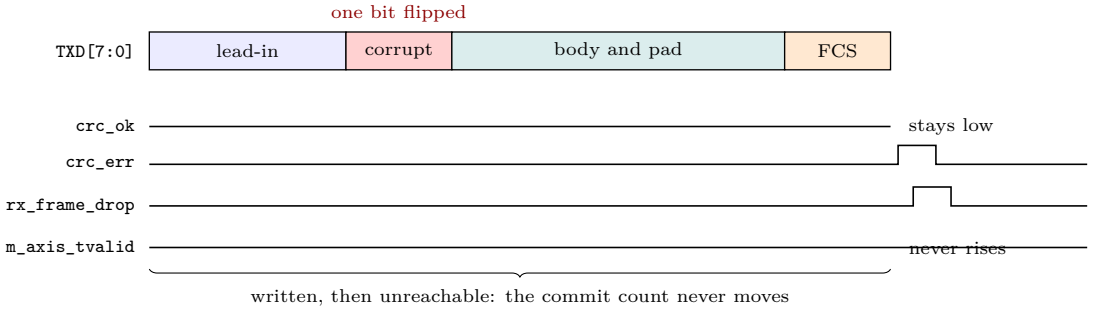


Fig. 5.3: The same bench with one corrupted octet in flight. The frame is still transmitted and still occupies the pins — corruption is not detectable at the transmitter — but the residue check fails, `rx_frame_drop` pulses, the commit count never advances and `m_axis_tvalid` never rises. The host therefore sees no frame at all, rather than a frame carrying a warning it might ignore.

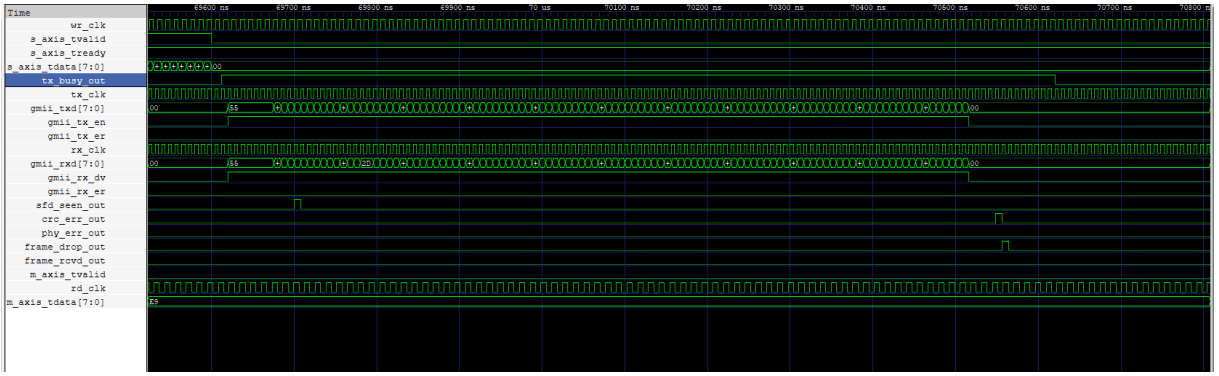


Fig. 5.4: Loopback waveform with one corrupted octet, from about 69.6 to 70.8 µs. `gmii_txd` and `gmii_rxd` start identically (`0x55` lead-in); `sfd_seen_out` pulses near 69.7 µs; near 69.78 µs one octet on `gmii_rxd` reads `0x2D`, the value damaged by the channel. When `gmii_rx_dv` falls near 70.6 µs, `crc_err_out` and then `frame_drop_out` pulse, `frame_rcvd_out` and `m_axis_tvalid` stay low, and `m_axis_tdata` still shows the `0xE9` of an earlier frame.

Fig. 5.4 is the recorded counterpart of that schematic. Nothing changes on the transmit side between the two runs — the same octets leave the pins, and the error cannot be seen there, because a transmitter cannot know a bit will be damaged after it leaves. The frame in this capture is longer than the three-octet example, but the behaviour is the same. Everything that distinguishes them happens at the receiver's decision point, four octets after the damage.

5.4 Defects Found

Three defects mattered because value-only checking passed all of them; each was caught by an assertion about *when* a signal was valid, not what it carried.

Table 5.2 lists them.

Table 5.2: The three defects that value-only checking passed, and what each looked like.

Defect	**Why value checks missed it**
Hold-back published a cycle late	Forty-one value checks passed; an idle cycle inside the frame broke the CRC, and only a contiguity assertion on `valid_out` saw it.
`frame_cmt` rebuilt from flags	Reconstructing the commit from `crc_ok` and the inverse overflow flag looked like a simplification, but the overflow flag is registered a cycle later, so the frames that should have been dropped counted as received.
A one-octet frame hung the padder	Start and end of frame assert on the same cycle, so no single signal looks wrong; the bench detects the stall.

5.5 Discussion

Three numbers summarise the outcome: 2,270 lines of RTL, **915 assertions, zero failures**. A little over two assertions per line is not exhaustive checking, but it is enough that every module has an assertion about its interface behaviour.

What the regression does not establish matters equally:

- **Simulation proves behaviour** only on the traces it drives;
- there is no coverage measurement and no formal property checking;
- there is no pin-level timing against a real PHY; and
- the clock crossings are *argued* rather than simulated, because Verilog has no metastability: the defence is structural, and the benches test the invariants that follow from it — no lost, duplicated or partial frame — rather than the structure itself.

Chapter 6. Conclusion and Future Work

6.1 Summary

This report asked a narrow question — can an Ethernet MAC that matters in practice be specified, built and proved from the public standard alone — and the answer is yes, with qualifications worth stating. A full-duplex IEEE 802.3 MAC was derived from the standard rather than from a reference implementation, written as fourteen vendor-neutral Verilog-2001 modules of about 2,270 lines, and verified by **915 self-checking assertions** that ran to completion with no failures. It replaces a core quoted at seven to eight lakh rupees per instance, at no licence cost and with no vendor primitive in the source.

Table 6.1 gives the numbers.

Table 6.1: Summary of what was built and verified.

Item	**Result**
Design	Full-duplex IEEE 802.3 MAC, GMII and MII, vendor-neutral Verilog-2001
Size	14 modules, about 2,270 lines of RTL
Clock domains	Three unrelated clocks, crossed by one Gray-coded frame count per FIFO
Verification	14 self-checking testbenches, 915 assertions, 0 failures
Defects found by structural assertions	3
Licence cost	None; replaces a core quoted at seven to eight lakh rupees per instance

One datapath serves GMII and MII, so the difference between a gigabit port and a hundred-megabit port is a strap rather than a second design. Three unrelated clocks are crossed by construction, so a frame is **wholly visible or wholly invisible** on the far side and a discarded frame leaves no trace to clean up. And the design demonstrates itself at the pins, where a three-octet payload becomes a legal sixty-octet frame and returns byte-identical, and where a single corrupted bit anywhere in flight produces no delivery at all.

6.2 What the Design Settles

The work also settles a smaller question that the literature leaves open. A MAC is usually described as a set of protocol functions, and if that description were complete a pipeline would suffice. It is not complete: the functions interact at exactly two places — the end of a frame on receive, where the last octet is known only after it has arrived, and the abort of a frame on transmit, where the wire must be drained before it can be reused — and both need state a pipeline does not have.

The answers are structural rather than procedural. The transmitter is a star with a hub, so preamble, checksum and gap timer are sources and timers rather than stages that can be reordered by accident; the receiver holds an octet back and publishes it on the edge that observes the end of the frame, so no bubble enters a stream whose checksum advances on every valid cycle. Neither choice is visible in a value trace, which is exactly why both were found by assertions about timing.

6.3 Limitations

The limits of the design are stated plainly:

- **Full duplex only:** no carrier sense, collision detection or backoff, so it cannot run on a shared medium.
- **Missing features:** no PAUSE handling, no MDIO, no address filtering, no VLAN tagging and no statistics block.
- **MII** is exercised against a behavioural model, not a real PHY.
- **No board bring-up**: no synthesis results, timing closure or pin-level sign-off are claimed.
- **Clock crossings** are argued rather than simulated, because a simulator has no metastability to exhibit.
- **Four minor gaps** found by the edge-case bench remain open, each documented with the bench that found it.

6.4 Future Work

The remaining work follows the order in which a user would need it:

1. **Half duplex** turns the transmitter from a star with a hub into a star with an arbiter: carrier sense, collision detection, the jam sequence and truncated binary exponential backoff all take the wire away from a frame already in progress, which is the one event this design never has to survive.
2. **Flow control**: PAUSE needs a receive path that parses a frame rather than merely checking it.
3. **A management interface**, which would turn the strap inputs into software configuration.
4. **Address filtering and VLAN tagging**, which need memory rather than counters, together with the verification that simulation cannot reach.
5. **Synthesis, timing closure and a board demonstration**, the step this report deliberately stops short of and the only test that can confirm its pin-timing assumptions. It is also the purpose of the project (Section 1.1): connect the MAC to the **MicroBlaze and DRAM interface**, drive the GTY transmitter with its internal protocol blocks bypassed, and use the known frames to test whether the surrounding XGS-PON hardware works.

Until then the claims above are claims about behaviour in simulation, and they are stated as such.

Chapter 7. Handover and Continuation

This chapter is written for the person who inherits the code rather than for the person who examines the report. The chapters above describe what the design is and why it is built that way; what follows records the things the source does not say about itself — which file is compiled with which, what a bench prints when it passes, which lines may not be edited without breaking a guarantee that no value trace would reveal, and exactly how far the work has been carried toward a board.

7.1 What Is Delivered

The delivery is fourteen synthesizable Verilog-2001 files together with the benches that prove them. Appendix A lists them in build order and separates the files that may be synthesized from those that may not. No vendor primitive appears anywhere, implicit nets are disabled in every file, every reset is asynchronous-assert active-low, every ready signal is derived from state and never from the matching valid, and every instantiation uses fully named port maps. Those four conventions are what let the same RTL move between simulators and synthesis tools; none of them should be relaxed for convenience.

Three facts about the code are worth stating before anything else, because each one is the natural assumption and each one is wrong.

1. **The verified top is not `ethernet_mac_top`.** The pin-to-pin loopback bench instantiates the transmit and receive controllers directly and wires the transmit pins to the receive pins. The combined wrapper elaborates cleanly and exports the status bus, but no bench drives it. This is an open item rather than a defect, and Section 7.5 treats it as the first thing to close.
2. **The transmitter is a star and the receiver is a line.** A block diagram that draws the transmit chain as checksum, padder, preamble, gap and pin stage in series is wrong: `tx_mac_controller` is the hub, the checksum is a tap on the padder's output, the preamble is a second source into the pin multiplexer, and the gap timer has no data ports at all (Section 3.2). The receiver, by contrast, is four stages in a row whose last member is the FIFO, wrapped by a controller that contains no logic (Section 3.4).
3. **Nothing below GMII exists here.** The block ends at `gmii_txd`, `gmii_tx_en` and `gmii_tx_er`, and it begins at `gmii_rxd`, `gmii_rx_dv` and `gmii_rx_er`. Coding, scrambling, serialisation, magnetics and the medium belong to the PHY (Section 1.1), and a task that appears to need them is a task about the board, not about this code.

7.2 Rebuilding the Evidence

A unit bench compiles the module under test and nothing else, so a failure names one file. The loopback bench compiles both chains and is the only bench that carries a whole frame from one end to the other. Icarus is invoked with `-g2005`, the language generation the benches are compiled against, while the RTL itself stays inside Verilog-2001; every bench writes its own `.vcd` for GTKWave and prints one line per assertion followed by a summary, so a run that prints nothing has hung rather than passed.

The commands below are the complete form, not a wrapper around them. A single module, using the checksum accumulator as the example:

```
cd TX
iverilog -g2005 -o sim_tx_fcs_tb.vvp tx_fcs_tb.v tx_fcs.v crc32_parallel.v
vvp sim_tx_fcs_tb.vvp
gtkwave tx_fcs_tb.vcd
```

The first source is always the bench and the rest is the design under test. The checksum engine must accompany any bench that reaches it, which means `tx_fcs`, `rx_fcs` and both controllers; `tx_edge_case_tb` additionally needs `tx_protocol_checker.v`, and `tx_fifo_tb` instantiates the pin stage, so those two compile together.

The whole design, end to end through the loopback channel:

```
iverilog -g2005 -o sim_loop.vvp ethernet_mac_loopback_tb.v \
    TX/tx_mac_controller.v TX/tx_fifo.v TX/tx_padding.v \
    TX/tx_fcs.v TX/tx_preamble.v TX/tx_ifg.v TX/tx_gmii_mii.v \
    TX/crc32_parallel.v \
    RX/rx_mac_controller.v RX/rx_gmii_mii.v RX/rx_preamble.v \
    RX/rx_fcs.v RX/rx_fifo.v
vvp sim_loop.vvp
```

The entire regression in one command, with one log per bench and a summary file beside them:

```
bash run_all.sh
```

That script prepends the oss-cad-suite binary directory to `PATH` on its fourth line, and that is the only line to edit if the tools live somewhere else. A per-module synthesis check, which establishes elaboration, inference and area but is not a timing sign-off:

```
cd TX
yosys -s synth.ys
```

The script reads one file, sets the top, runs `synth`, prints statistics and writes a netlist. Extending it to another module means changing the read line and the top; it is a sanity check, not a constraint set.

7.3 The Interface in One Page

Almost every integration question is answered by Table 7.1: what the host may assume, what the PHY must supply, and which signal belongs to which clock.

Table 7.1: The interface contract of the combined core.

Group	Clock	What the other side may assume
Host transmit	`host_clk`	AXI4-Stream slave, eight bits. `tready` is derived from FIFO state only; `tlast` commits the frame, so a partial frame can never reach the wire. Any length from one octet upward; the MAC pads
Host receive	`host_clk`	AXI4-Stream master, eight bits, `tlast` on the final body octet. Only complete, checksum-good, length-legal frames appear, and the pad is included
PHY transmit	`gtx_clk`	Registered outputs; one octet per clock in GMII, low nibble then high in MII, with `TXD[7:4]` driven zero
PHY receive	`rx_clk`	No ready and no backpressure. `rx_clk` belongs to the PHY and is never generated by this block except in loopback
Configuration	strap	`speed_select` and `loopback_en` are reset-time straps, not runtime controls
Status	mixed	Fifteen raw outputs. Eleven are one-cycle pulses in the domain that produces them; the rest are levels, or are valid only with the frame's last octet

The status bus deserves one sentence of method. A one-cycle pulse is shorter than a clock period of any slower domain, so synchronising it loses it silently: count the pulses in the domain that produces them and cross the counter. The two levels are the exception, and `tx_busy_out` deliberately includes the inter-frame gap, which makes the strap contract observable from a single bit. Appendix A lists all fifteen with their domains and meanings.

7.4 Rules That Carry the Design

These are the places where a plausible edit produces something that still simulates, still frames correctly, and is wrong. They are ordered roughly as a newcomer meets them.

1. **The checksum is one octet ahead of the wire.** The checksum engine is tapped at the input of a single lead register, so it consumes each octet one cycle before the pin stage does and its result is ready in exactly the cycle the four trailer octets must begin. That is zero slack, not generous margin. Adding or removing one stage between the padder and the pin stage leaves every frame with a well-formed and wrong trailer, and nothing else in the design looks different.

2. **The checksum tap closes on the last octet.** The padder returns to idle in the same cycle in which it presents its end-of-frame marker, and it is ready there. Without the extra term in its ready expression the next frame's first octet enters the checksum one cycle before the final value is latched, and the shipped trailer covers one octet of the following frame. The padder's freeze branch then holds that octet until the next launch, which is why closing the tap costs nothing.

3. **The gap has two one-cycle holes.** The pin stage is ready throughout its idle state, including the whole gap, so its ready signal cannot decide when to launch. Two windows must be closed by the controller: the first cycle after the carrier falls, before the gap timer's level has risen, and the cycle in which the transmitter's own last trailer octet is still inside the pin stage (Section 3.3). Gating on the timer's level rather than its done pulse is deliberate — it is the exact legal minimum. The gap actually measured at the pins is 120 bit times in GMII and 108 in MII against the 96-bit-time floor, because the pin stage's forced end-of-frame cycle and the launch decision add idle time of their own.

4. **Only a frame count crosses a clock boundary.** A Gray code is safe for a quantity that changes by one and never rewinds. The committed frame counts satisfy that; the octet pointers do not, because they jump per octet, advance by a whole frame on commit, and rewind on a dropped frame. So only the count crosses, through two flops, and the reader derives the end of a frame from the marker stored with the octet itself. This is also why a rewound frame is invisible to the far side: the count never reached it. Any change here is a silicon-risk change, because a simulator has no metastability to show.

5. **Receive has no bubble, ever.** The receive checksum advances on every valid octet, so a single idle cycle inside a frame corrupts the digest even though all the octets are right. The pin stage therefore holds one octet back and publishes it on the same edge that observes the carrier fall, not in the following state. The version that published one cycle late passed every value check and was caught only by an assertion that valid stayed high from the first octet to the last.

6. **The verdict arrives one cycle after the data.** The final trailer octet enters the checksum on the edge that ends the frame, so the receiver cannot know whether the frame is good until the next cycle. That single cycle is why the receive FIFO commits rather than streams, and why no plain valid/ready FIFO can take its place.

7. **A rejected frame is rejected whole.** The receive FIFO rewinds to its committed pointer, so a bad frame is never partly visible and there is nothing to undo. Overflow behaves the same way and outranks a good checksum: a frame that could not be stored cannot be delivered. The commit pulse comes from the commit branch itself; reconstructing it as "checksum good and not overflow" is one cycle early and counts an overflowed frame as received, which was a real defect caught by the loopback bench.

8. **The pad is delivered.** A three-octet payload returns as sixty octets. Recovering the true length is the client's job, from the Length/Type field; a MAC that stripped the pad would have to parse that field and would sometimes guess wrong, which is worse than a known-wrong length.

Two smaller traps belong with them. A one-octet frame is legal input, and it needs the padder's parked output in the launch condition — testing the FIFO's frame flag alone loses it forever. And the in-flight

capacity of each FIFO is one word more than its depth, because the first octet is prefetched; that is not an off-by-one, and sizing for exactly one maximum frame would underrun.

7.5 What Is Proven and What Is Not

Table 7.2 is deliberately blunt. It is the table a successor should read before planning anything, because the distance between the two halves of it is the whole of the remaining work.

Table 7.2: What has been established and what has not.

Area	State
Transmit datapath	Complete and verified: preamble, padding, checksum, gap, GMII and MII pin stage, underrun abort
Receive datapath	Complete and verified: deserialisation, delimiter strip, checksum verify and strip, length check, commit or discard
Clock crossings	Two asynchronous FIFOs, one Gray count each, per-domain reset merge; argued structurally, because simulation cannot exhibit metastability
Combined wrapper	Written and elaborates cleanly; no bench drives it
Loopback	Pin to pin, end to end, including the corruption sweep
MII at the top level	Verified per module and on the transmit side; no top-level MII round trip
Synthesis and timing	Per-module elaboration and area only; no closure, no I/O timing, no crossing sign-off
Board	Nothing. No PHY, magnetics, cable or coding core has been driven

The first item to close is the missing wrapper bench, because it is cheap and because everything in the next section depends on it: instantiate the wrapper, enable internal loopback, drive a three-octet frame through the host port, and assert the same round trip the present bench asserts — this time through the actual wrapper and its multiplexer.The four gaps recorded by the edge-case bench (Chapter 5) stay open and are documented rather than hidden.

7.6 Six Things Not To Do

Each of these has a specific consequence, and none is a style preference.

1. Do not connect `fcs_valid`, `sfd_done_out` or `ifg_done`. They are unconnected on purpose: the checksum strobe fires on every bubble in the stream, and the other two are one cycle later than the fact the controller needs.
2. Do not put logic, and in particular not a reset, inside the FIFO memory processes. Both arrays are reset-free so that they infer block RAM; a reset there turns two thousand words into eighteen thousand flip-flops.
3. Do not merge the three resets. Each domain releases its own after its clock is stable. Each FIFO already merges its own two sides internally, which is why resetting one port clears both, and why asserting one reset while traffic is in flight loses the other side's contents.
4. Do not synchronise a status pulse. Count it in its own domain and cross the count.
5. Do not drive `rx_clk` from anything except the PHY — or, in internal loopback, from `gtx_clk` through a clock multiplexer that is constrained as one. There is no PHY sourcing it in loopback, and a free-running clock samples the looped-back frame at the wrong rate.
6. Do not assert `TX_ER` while `TX_EN` is low. The standard assigns that combination to carrier extend and its error, not to idle, so a PHY is entitled to treat the gap as a fault.

Two facts about the surrounding system follow from the same discipline. The MAC has no management interface, so a PHY comes out of reset unconfigured unless something else configures it. And `speed_select` may only change while `tx_busy_out` is low; in MII the transmit clock's *direction* also changes, so a live speed change is a board reconfiguration and not a register write.

7.7 Continuing to Hardware

The MAC is a GMII client. On a board it meets either an external PHY, with GMII pins on the FPGA, or an internal coding and serialising core that consumes GMII and drives a transceiver. Either way the MAC's own pins do not change, and the work divides into a wrapper, a constraint set, one missing bench and a bring-up order.

1. **Choose the PHY path** and the speeds it must support. One gigabit GMII is the verified mode; MII is supported but less proven at the top level.
2. **Write the board wrapper.** It owns the reference clock and the PLL, the transmit clock and its forwarding to the PHY, the receive clock input, the host clock, reset sequencing, the loopback clock multiplexer, the speed strap and the I/O buffers. No vendor primitive belongs inside the MAC.
3. **Add constraints.** Three clock definitions, source-synchronous input and output delays on both GMII directions, pin assignments and I/O standards, and asynchronous constraints on the two Gray buses — keeping the synchroniser attributes so no stage is optimised away.
4. **Write the wrapper bench** before synthesis, as above.
5. **Synthesise and close timing.** Check that both FIFO arrays infer block RAM and that no latches appear, then that the 125 MHz paths close. The reset-free memory ports exist for this check.
6. **Bring up in stages**: internal loopback on the board with the PHY held in reset, then the PHY link, then external loopback, then real traffic from a host. A failure in the first stage is a MAC, clocking or reset fault, and saying so is the point of having the stage at all.

Table 7.3 gives the acceptance test for each stage, so that "it works" means something specific at every step.

Table 7.3: Acceptance tests for the hardware continuation.

Stage	Acceptance
M1 — wrapper bench	The wrapper with loopback enabled and the receive clock taken from the transmit clock round-trips a three-octet frame as sixty octets; the sent, received and available status signals all behave
M2 — synthesis	Both FIFOs infer RAM, no latches appear, timing closes at 125 MHz, crossing constraints in place
M3 — board loopback	The same round trip on hardware with the PHY ignored and the status counters readable
M4 — PHY link	Link up at one gigabit, clean GMII in both directions, external loopback at the PHY passing
M5 — traffic	Ping and ARP from a host, short and maximum-length frames, and a deliberately corrupted frame dropped without reaching the host

In this project the immediate target is the first of the two PHY paths: a bridge from the MAC's GMII to the serial gigabit interface of the transceiver on the board, and beyond it the transceiver-based optical path that motivated the work (Section 1.1). The bridge, not the MAC, owns the serial side, its reference clock and its own line coding, so the MAC continues to see a 125 MHz octet interface and no RTL above the pins changes. What does change is the wrapper, because the loopback clock selection, the transmit clock forwarding and the reset sequencing all move into the bridge's domain. That is the reason for keeping the target generic in the design and specific only in the wrapper: a different PHY or a different board then costs a wrapper, not a redesign.

Until a board run exists, every claim in this report remains a claim about simulation. The shortest path from here to a claim about hardware is the wrapper bench, and it is the first thing a successor should build.

Chapter 8. Bibliography

Bybell, G. R. (2023). GTKWave electronic waveform viewer. `http://gtkwave.sourceforge.net`.

Cummings, C. E. (2002). Simulation and synthesis techniques for asynchronous FIFO design. In *Synopsys Users Group (SNUG) Conference*, San Jose, CA, USA.

Cummings, C. E. (2008). Clock domain crossing (CDC) design and verification techniques using SystemVerilog. In *Synopsys Users Group (SNUG) Conference*, San Jose, CA, USA.

IEEE Computer Society (2018). *IEEE Standard for Ethernet, IEEE Std 802.3-2018*. Institute of Electrical and Electronics Engineers, New York, NY, USA. Clauses 3, 4, 22, 35 and 36.

Intel Corporation (2021). *Triple-Speed Ethernet Intel FPGA IP User Guide, UG-01008*. Intel Corporation, San Jose, CA, USA.

Koopman, P. and Chakravarty, T. (2004). Cyclic redundancy code (CRC) polynomial selection for embedded networks. In *International Conference on Dependable Systems and Networks (DSN)*, pages 145–154.

OpenCores (2012). Ethernet MAC 10/100 Mbps (ethmac). `https://opencores.org/projects/ethmac`.

Williams, S. (2023). Icarus Verilog compiler, release 14.0. `http://iverilog.icarus.com`.

Xilinx Inc. (2017). *Tri-Mode Ethernet MAC LogiCORE IP Product Guide, PG051, version 9.0*. Xilinx Inc., San Jose, CA, USA.

Appendix A. Delivery Reference

This appendix is the map to the code. Paths are relative to the project root, where `TX/` and `RX/` hold the sources and `Chapters/` and `Pictures/` hold this report. The RTL is the authority; where the two disagree, the RTL is right and this appendix is stale.

A.1 File Inventory and Build Order

Fourteen files form the synthesizable set. Icarus does not require a particular order, because there are no preprocessor dependencies between them, but the list below is in dataflow order so that a build script reads top to bottom.

```
TX/ethernet_mac_top.v      combined core (wrapper, loopback mux)
TX/tx_mac_controller.v     transmit hub, seven-state machine
TX/tx_fifo.v               frame-aware asynchronous FIFO
TX/tx_padding.v            minimum frame length
TX/tx_fcs.v                checksum accumulator and latch
TX/tx_preamble.v           preamble and delimiter source
TX/tx_ifg.v                inter-frame gap timer
TX/tx_gmii_mii.v           transmit pin stage
TX/crc32_parallel.v        one-octet checksum step (combinational)
RX/rx_mac_controller.v     receive wrapper, no logic
RX/rx_gmii_mii.v           receive pin stage, hold-back octet
RX/rx_preamble.v           delimiter search
RX/rx_fcs.v                checksum verify, strip and length check
RX/rx_fifo.v               store-and-forward frame gate
```

Everything else is test material and must not be synthesized: every `*_tb.v`, and `tx_protocol_checker.v`, which exists only to be instantiated by the edge-case bench. Two others are easy to get wrong: the file `TX/crc32_parallel.v` has an identical twin in `RX/`, and the two must stay identical (or only one may appear in a build list), because the transmitter and the receiver agree on a checksum only if they run the same engine. And the transmit FIFO bench instantiates the pin stage, so those two compile together.

A.2 Parameter Reference

Table A.1: Parameters, their defaults and the constraints that make the defaults correct.

Parameter	Where	Default	Constraint
`DATA_W`	top, controllers, FIFOs	8	Must stay 8; several ports are hard-coded octet-wide
`TX_FIFO_ADDR_W`	top, transmit side	11	Depth must exceed the largest frame; 2048 against 1514 leaves headroom
`RX_FIFO_ADDR_W`	top, receive side	11	Same rule as the transmit side
`MIN_FRAME_LEN`	top, controllers, padder, receiver	60	Sixty-four on the wire less the four trailer octets; keep transmit and receive equal
`MAX_FRAME_LEN`	top, receiver	1514	1518 less the trailer; raise together with the FIFO depth for tagged or jumbo frames
`PREAMBLE_LEN`	top, transmit controller, preamble source	7	The delimiter is the eighth octet
`IFG_BIT_TIMES`	top, transmit controller, gap timer	96	The standard's minimum; 12 cycles in GMII, 24 in MII
`ERR_CYCLES`	top, transmit controller, pin stage	2	Legal range 1 to 4; guarded at elaboration
`PREAMBLE_BYTE`	top, receiver	`0x55`	The standard's preamble octet
`SFD_BYTE`	top, receiver	`0xD5`	The delimiter as received, low bit first
`PRE_MAX`	top, receiver	32	A sanity bound on a stuck link, not a specification limit
`INT_LOOPBACK`	top	1	Zero removes the multiplexer; tie the enable low if removed

A.3 Status and Configuration Signals

Fifteen status outputs leave the core. Eleven of them are one-cycle pulses in the domain listed and lose their meaning if synchronised; four are not, and are marked as such in Table A.2. The two configuration inputs are straps, sampled but never used as runtime control.

Table A.2: Status and configuration signals. Domain determines how a host must capture the signal.

Signal	Domain	Type	Meaning
`tx_frame_drop_out`	host	pulse	A host frame was never sent, because the FIFO overflowed or deadlocked
`tx_frame_sent_out`	wire	pulse	The pin stage finished a frame; the carrier has fallen
`tx_underrun_out`	wire	pulse	The upstream underran, and the frame was deliberately corrupted and drained rather than truncated
`tx_busy_out`	wire	level	A frame is in flight *or* the gap is counting
`rx_frame_rcvd_out`	receive	pulse	A frame was committed for the host
`rx_frame_drop_out`	receive	pulse	A frame was lost: bad checksum, bad length, PHY error or overflow
`rx_overflow_out`	receive	pulse	The drop was an overflow, one cycle after the drop
`rx_crc_err_out`	receive	pulse	Bad checksum, with length and PHY errors folded in
`rx_len_err_out`	receive	pulse	A runt or an oversize frame; the reason, not the gate
`rx_phy_err_out`	receive	pulse	The PHY flagged the frame; the reason, not the gate
`rx_no_sfd_out`	receive	pulse	The carrier ended without a valid delimiter, so no frame existed to check
`rx_sfd_seen_out`	receive	pulse	The delimiter was consumed and a frame has begun
`rx_frame_ok_out`	receive	with last	This carrier contained a real, delimited frame
`rx_frame_len_out`	receive	level	Body octets published, including pad and excluding the trailer, stable with the verdict
`rx_frame_avail_out`	host	level	At least one committed frame is waiting or streaming
`speed_select`	strap	input	Zero selects nibble width, one selects octet width
`loopback_en`	strap	input	Substitutes the transmit pins for the receive pins

A.4 Clause Map

Table A.3 is the shortest answer to "where is this requirement implemented", and the reason the report cites so few clauses elsewhere: each one is answered in a named file rather than in prose.

Table A.3: Clauses of the standard and the file that implements each.

Clause	**Subject**	**Implemented in**
3.1.1	Frame format	both controllers
3.2.1, 3.2.2	Preamble and delimiter	preamble source, delimiter search
3.2.9	Checksum coverage and transmission order	checksum accumulator, transmit controller, receiver
3.3	Low-bit-first order	both pin stages, delimiter value
4.2.3.3	Uninterrupted transmission, maximum frame	transmit FIFO, transmit pin stage
4.2.4.1.2	Invalid frames must not reach the client	receive FIFO
4.2.4.2.1	Tolerate a shortened preamble	delimiter search
4.2.5	Preamble and delimiter are not part of the frame	transmit controller, delimiter search
4.4.2	Inter-frame gap, minimum and maximum frame size	gap timer, padder, receiver
22.2.2, 22.2.3	MII signals and nibble order	both pin stages
35.2	GMII signals and the transmit enable table	both pin stages